

CAMPUS VIRTUAL TRAINING

CVTScript v3

Manuale per Principianti

Come scrivere un tutorial passo-passo,
spiegato semplice

Versione del linguaggio:	3.0 (completa)
Data:	Maggio 2026
A chi serve:	A chi non sa programmare
Tempo di lettura:	50–60 minuti

Leggi una volta, prova subito, divertiti.

Per cominciare

Questo manuale ti insegna a scrivere un **tutorial 3D** per Campus Virtual Training **senza saper programmare**. Lo leggi una volta, ti diverti, e alla fine sei capace di creare le tue lezioni interattive.

Cosa NON serve. Non serve conoscere nessun linguaggio di programmazione. Non serve saper usare strumenti tecnici. Non serve avere studiato informatica.

Cosa serve invece. Serve sapere *cosa* vuoi insegnare allo studente, passo dopo passo. Esattamente come quando spieghi a un amico come fare una torta: prima si prende la ciotola, poi si rompono le uova, poi si aggiunge la farina. . .

Come è organizzato il manuale. Sono **capitoli brevi**, divisi in tre parti. La **prima parte** (cap. 1–10) spiega le **basi**: ti basta per scrivere un tutorial completo. La **seconda parte** (cap. 11–17) spiega le **cose avanzate** (oggetti in mano, finestre con messaggi, animazioni, oggetti interattivi). La **terza parte** (cap. 18) spiega i **file di configurazione** che vivono attorno al tutorial. Alla fine c'è un **glossario** con tutte le parole importanti tradotte in italiano semplice. Se ti perdi, torna al glossario.

Come è organizzato il manuale

Il manuale è diviso in tre parti. Segui l'ordine la prima volta; poi consulta a salti ciò che ti serve.

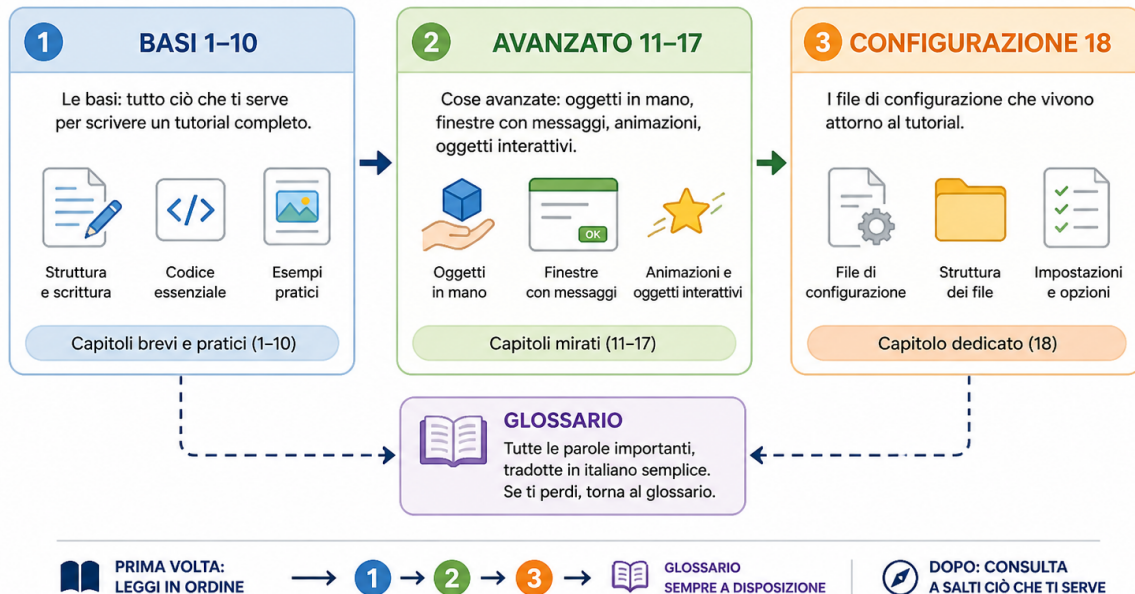


Figura 1: Le tre parti del manuale: leggile in ordine la prima volta, poi consulta a salti.

Figura 1: Le tre parti del manuale: leggile in ordine la prima volta, poi consulta a salti.

Buona lettura!

Indice

1	Cos'è un tutorial?	1
2	I modelli 3D: la scena dove tutto accade	4
2.1	Il formato richiesto: .glb	4
2.2	Esportazione da Blender	5
2.2.1	Convenzioni di denominazione	8
2.2.2	Oggetti compositi con figli interattivi	8
2.3	Origine, direzione e posizione iniziale	9
2.3.1	Origine (pivot)	9
2.3.2	Direzione di movimento	10
2.3.3	Posizione iniziale	11
3	Organizzare il progetto: cartelle e file	12
3.1	Struttura delle cartelle del progetto	12
3.2	Registrare i modelli in homeconfig.ini	13
3.3	Checklist prima di iniziare a scrivere	14
4	Come è organizzato un file cvtscript	16
4.1	La forma di una riga	16
4.2	La gerarchia a tre livelli	17
4.2.1	Livello 1 – [scene]: la cornice globale	17
4.2.2	Livello 2 – [section "Titolo"]: il capitolo	18
4.2.3	Livello 3 – [step "Titolo" flag]: il singolo passo	18
4.3	Altri blocchi accessori	19
5	Dentro uno step	20
6	Gli ordini, l'istruzione do :	22
6.1	Quando parte un do : ?	22
6.1.1	Le tre eccezioni: auto, machine, e l'attrezzo	22
6.1.2	Più do : in fila = una sola animazione	23
6.2	Il vocabolario degli ordini	24
6.3	Spiegazioni semplici di ognuno	24
6.3.1	unscrew – Svita la vite	24
6.3.2	screw – Avvita la vite	25
6.3.3	extract / insert – Estrae / Inserisce	25
6.3.4	place – Appoggia a terra	25

6.3.5	move – Sposta in linea retta	26
6.3.6	rotate – Gira	26
6.3.7	pump – Pompare	27
6.3.8	idle reset – Resetta tutto	28
7	Gli strumenti: i vari attrezzi	29
8	La camera: dove guarda lo studente	31
8.1	Le parti della camera	31
9	Trascinare gli oggetti (drag & drop)	33
9.1	Più oggetti contemporaneamente	33
9.2	Punto di aggancio personalizzato	33
10	Pulsanti, porte, leve: tutto è element	35
10.1	Caso A – Click su un pulsante che cambia uno schermo	36
10.2	Caso B – Click su una porta che si apre	36
10.3	Caso C – Click su una vite con un attrezzo	36
10.4	after con effetti multipli	37
10.5	La tabella che decide tutto	37
10.6	Click ripetuti: repeat N	38
11	Le bandierine auto, machine, highlight	39
12	Tenere oggetti in mano (hold)	41
12.1	hold = pick – Prendi in mano un oggetto	41
12.2	hold = held – Lo step richiede l'oggetto già in mano	42
12.3	hold = release – Rimetti giù	42
13	Mostrare un messaggio sullo schermo	43
14	Mettere a posto i pezzi prima di iniziare	45
14.1	position – Mettere un pezzo in un punto	45
14.2	rotation – Ruotare un pezzo	45
14.3	Dove si scrivono?	45
15	Pezzi che si muovono insieme (companion)	47
15.1	companion = X follows master – Mi attacco al master	47
15.2	companion = X moves by (...) duration t – Movimento separato	48
16	Schermate di un display 3D (view)	49
17	Animazioni 2D a frame (animation)	51
18	I blocchi [state] e [object]	53
18.1	[state nome] – Le varianti di un oggetto	53
18.2	[object nome] – Un oggetto con pulsanti dentro	54
19	I file di configurazione attorno al tutorial	56
19.1	homeconfig.ini – La vetrina degli scenari	56

19.2	<code>tool.ini</code> – Gli attrezzi della cassetta	58
19.3	<code>objects.ini</code> – I pulsanti e gli schermi della scena	59
19.4	<code>interfaceconfig.ini</code> – Personalizzare i comandi	61
20	Un esempio completo, riga per riga	62
20.1	Step automatici in catena	63
20.2	Esempio di drag & drop	64
20.3	Esempio di click su un pulsante 3D	64
20.4	Esempio di click su una porta	65
21	Errori comuni e come evitarli	66
21.1	Errore 1: scrivere le righe con la lettera maiuscola	66
21.2	Errore 2: dimenticare le virgolette nel nome dello step	67
21.3	Errore 3: numerare le azioni manualmente	67
21.4	Errore 4: scrivere il nome del file <code>.glb</code>	67
21.5	Errore 5: usare i nomi italiani degli attrezzi	68
21.6	Errore 6: dimenticare gli spazi attorno all' '='	68
21.7	Errore 7: mettere commenti con <code>//</code>	68
21.8	Errore 8: usare <code>AutoSetVariant</code> invece di <code>after</code> :	69
21.9	Errore 9: usare <code>ActiveButtons</code> + <code>AcceptTrigger</code> a mano	69
21.10	Errore 10: scrivere <code>button = ...</code> (vecchia versione del v3)	70
21.11	Errore 11: usare <code>Animated...</code> invece di <code>animation</code>	70
22	Glossario delle parole importanti	71
	Conclusione	73

Capitolo 1

Cos'è un tutorial?

Pensa a quando insegni a un amico **come fare una torta**. Non gli dici tutto in una volta. Vai per piccoli passi:

- **Passo 1:** prendi una ciotola.
- **Passo 2:** rompi due uova dentro.
- **Passo 3:** aggiungi la farina.
- ...

Un tutorial nel programma Campus Virtual Training funziona **esattamente così**, ma invece della torta lo studente sta imparando a **smontare una pompa, cambiare un filtro o lubrificare un elettromandrino**.



Figura 1.1: Stessa logica, due mondi diversi: la metafora della ricetta funziona perché ogni tutorial è una sequenza di passi semplici.

Tu, nel file `tutorial.cvtscript`, scrivi la lista dei passi. Il programma li esegue uno alla volta e fa muovere i pezzi sullo schermo.

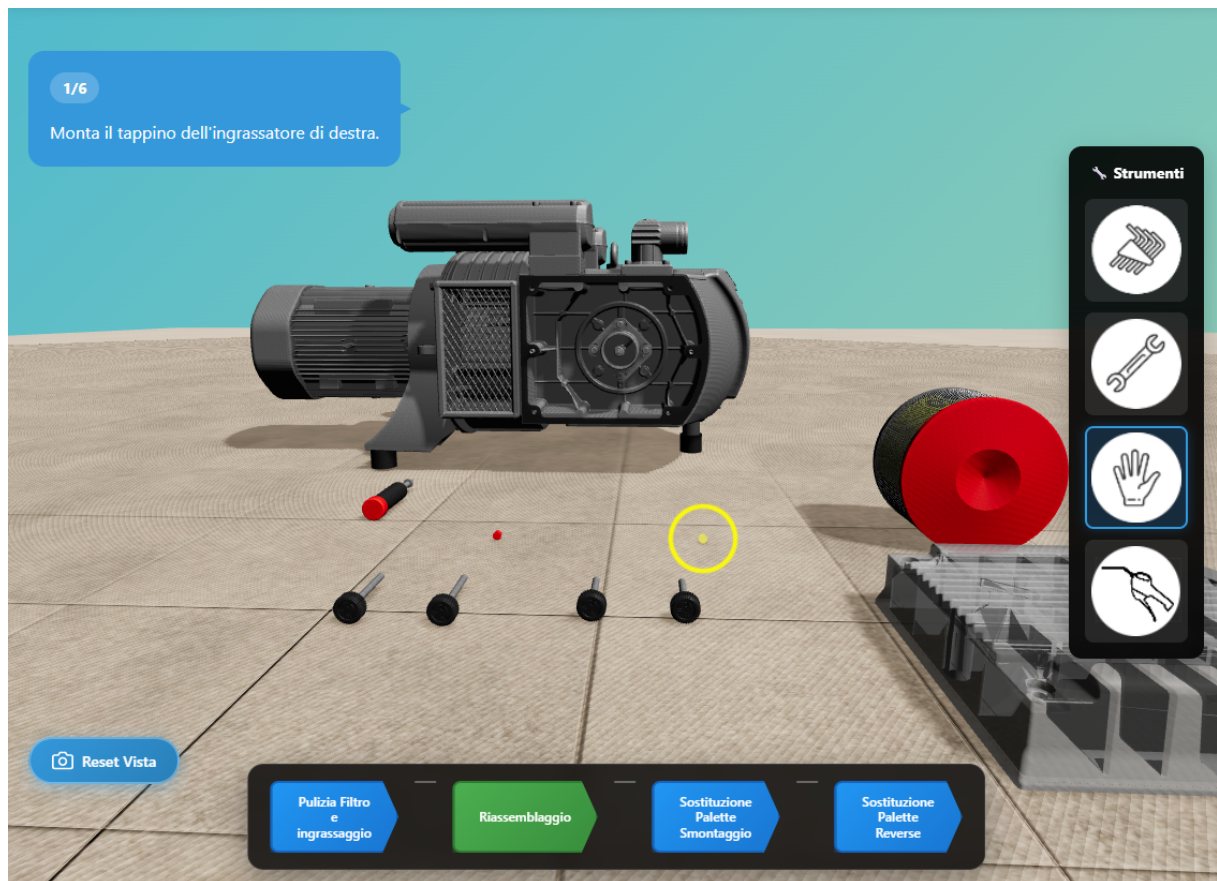


Figura 1.2: L'interfaccia che vede lo studente:.

In poche parole

Tu sei l'**autore di una ricetta**. Lo studente è un **cuoco apprendista**. Il programma è il **cuoco esperto** che dimostra ogni passaggio. Tu scrivi la ricetta in modo chiaro, e il programma ci pensa a far vedere tutto.

Capitolo 2

I modelli 3D: la scena dove tutto accade

Adesso che sai cos'è un tutorial, c'è una cosa da capire prima di iniziare a scrivere il tuo primo file: i pezzi che lo studente **vede e manipola** sullo schermo sono **modelli 3D**, ovvero oggetti tridimensionali caricati dal programma. Non li crei tu scrivendo il tutorial — esistono già come file separati. Tu nel tutorial dici solo *“prendi quella vite, svitala, mettila giù”* e il programma sa già come è fatta la vite perché l'ha caricata all'avvio.

Puoi saltare questo capitolo?

Se qualcun altro ha già preparato i modelli 3D e sai solo che si trovano nella cartella `models/`, puoi passare direttamente al Capitolo 4. Torna qui quando vuoi capire come vengono creati o se devi aggiungerli tu.

2.1 Il formato richiesto: .glb

Campus Virtual Training usa **Three.js** come motore di rendering e carica esclusivamente modelli in formato **glTF Binary** (.glb). È uno standard aperto e offre tre vantaggi pratici:

- **File singolo:** geometria, materiali e texture sono contenuti in un unico file binario.
- **Compatto:** la codifica binaria riduce le dimensioni rispetto a formati testuali come OBJ o COLLADA.
- **Compatibilità web:** è il formato nativo di Three.js e non richiede conversioni a runtime.

Ricorda

Se hai modelli in altri formati (.step, .stl, .obj, .fbx), devi convertirli in .glb prima di poterli usare. **Blender** è lo strumento consigliato per la conversione.

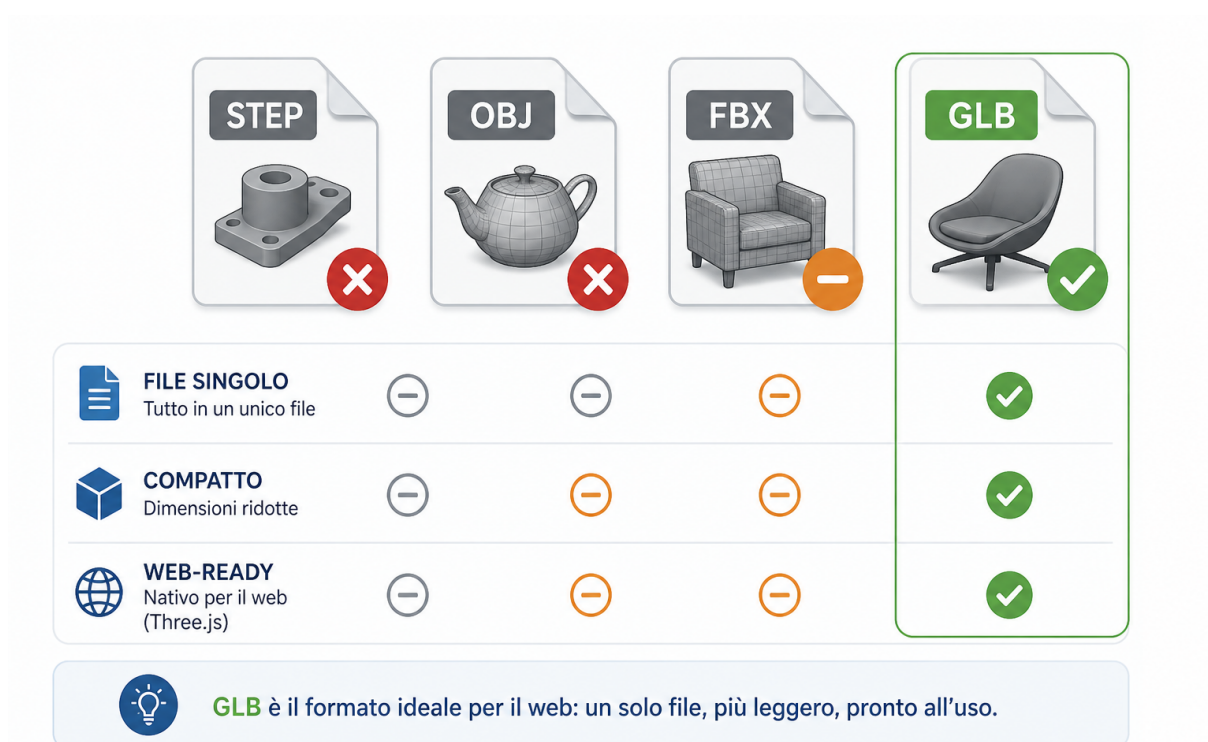


Figura 2.1: Perché GLB: un solo file, dimensioni ridotte, nessuna conversione a runtime.

Figura 2.1: Perché GLB: un solo file, dimensioni ridotte, nessuna conversione a runtime.

2.2 Esportazione da Blender

Blender è il software consigliato per preparare i modelli 3D. La procedura tipica è:

1. **Importa** il modello CAD originale (STEP, IGES o STL sono i più comuni nelle officine meccaniche).
2. **Pulisci la mesh**: elimina facce duplicate, correggi le normali invertite e riduci la complessità poligonale dove possibile.
3. **Assegna i materiali**: ogni materiale definito in Blender verrà esportato nel GLB. Usa nomi significativi.
4. **Posiziona l'origine**: l'origine dell'oggetto in Blender corrisponde al **pivot** attorno al quale Campus Virtual Training calcolerà rotazioni e traslazioni.
5. **Esporta** con *File* → *Export* → *glTF 2.0 (.glb/.gltf)*, selezionando il formato *.glb*. Nel pannello di esportazione, sotto la sezione *Geometry*, abilita l'opzione *Compression* (compressione **Draco**). Riduce le dimensioni dei file fino al 60–80 % — molto utile per scenari con molti modelli come la Pompa Becker (oltre 40 file GLB). Impostazioni consigliate:
 - *Compression Level*: 6 — buon compromesso tra dimensione e velocità di decodifica.
 - *Quantization Position*: 14 bit — precisione sufficiente per componenti meccanici.

- *Quantization Normal*: 10 bit.

Convenzione opzionale per i materiali emissivi

Se nel modello hai un materiale Blender che rappresenta un elemento realmente *auto-illuminante* (LED, spia luminosa, display retroilluminato) e vuoi che il sistema ne potenzi la luminosità in scena, puoi nominare quel materiale con un nome che contenga luminoso, emissive oppure glow: al caricamento il modello verrà rilevato e impostato a `emissiveIntensity = 3.0`.

È una convenzione *opzionale* sui nomi dei materiali, non una direttiva del file `cvtscript`, e nessuno degli scenari attuali la usa. Non confonderla con l'evidenziazione gialla degli elementi interattivi: quella è gestita automaticamente dal motore in base allo step e funziona indipendentemente dal nome del materiale.

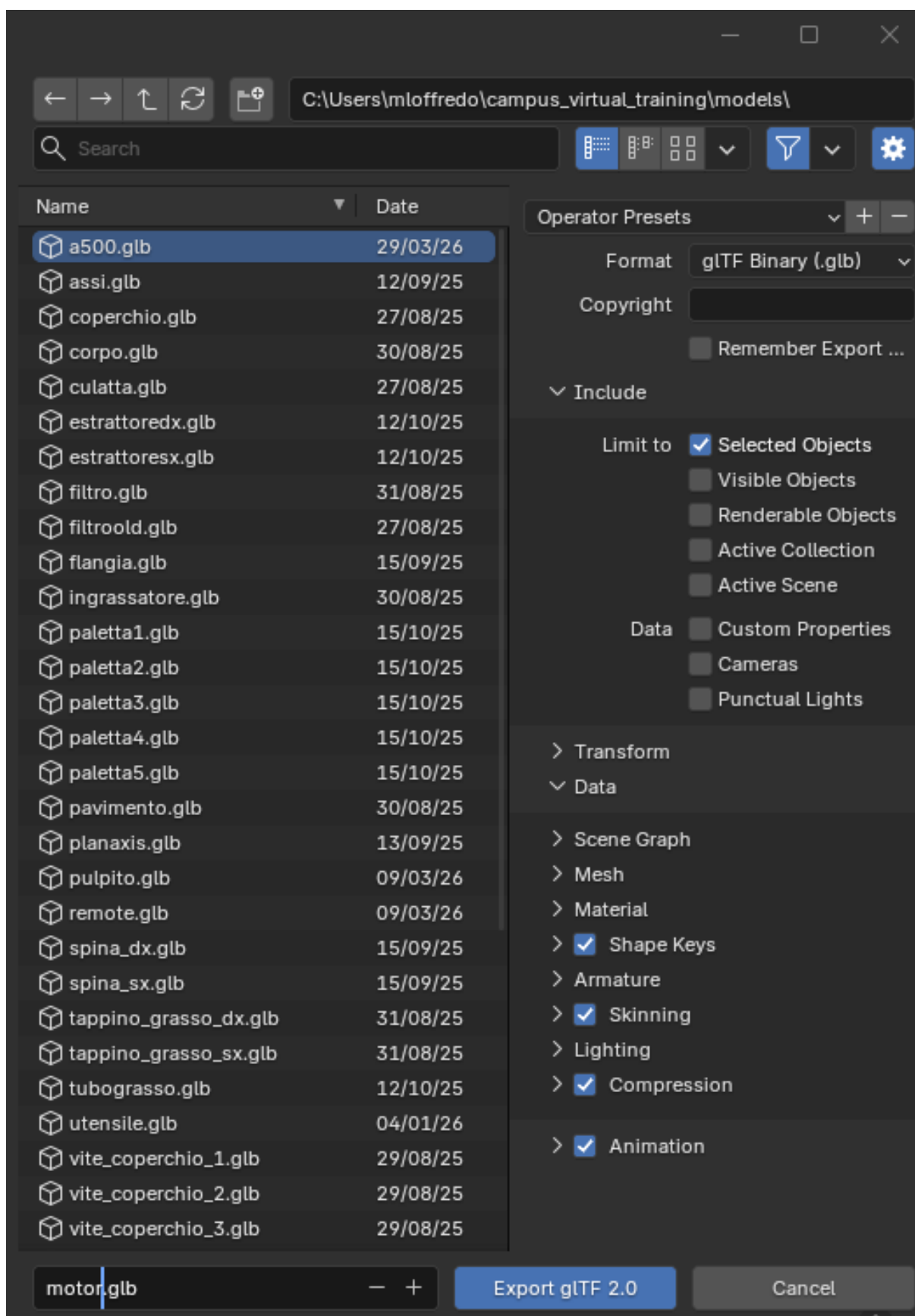


Figura 2.2: Il pannello di esportazione di Blender. La compressione Draco si trova sotto Geometry → Compression.

2.2.1 Convenzioni di denominazione

La denominazione dei file è fondamentale. Ogni file GLB deve avere un nome **univoco**, **tutto in minuscolo** e **senza spazi**. Per gli scenari della Pompa Becker, ad esempio:

- **Componente principale:** `corpo.glb` — il corpo della pompa, elemento statico di riferimento.
- **Coperchio:** `coperchio.glb` — il coperchio che verrà smontato e rimontato durante il tutorial.
- **Parti interne:** `filtro.glb`, `culatta.glb`, `flangia.glb` — componenti accessibili dopo lo smontaggio.
- **Viteria:** `vite_coperchio_1.glb`, `vite_coperchio_2.glb`... — ogni vite è un file separato per poterla animare indipendentemente.
- **Ambiente:** `pavimento.glb` — base statica della scena per dare contesto visivo.

Ogni oggetto che lo studente deve manipolare direttamente (svitare, estrarre, trascinare) deve essere esportato come file `.glb` autonomo. I sotto-assiemi che si muovono *automaticamente* (per esempio gli assi di una macchina) possono invece restare come figli all'interno del GLB principale e venire animati tramite la notazione puntata `element = a500.porta`.

[Immagine da inserire]

Screenshot dell'Esplora risorse (o Finder) aperto sulla cartella `models/` della Pompa Becker. Si vedono i file GLB con nomi minuscoli: `corpo.glb`, `coperchio.glb`, `filtro.glb`, varie `vite_*.glb`, `pavimento.glb`.

Figura 2.3: La cartella `models/` della Pompa Becker: ogni componente smontabile è un file GLB autonomo.

2.2.2 Oggetti composti con figli interattivi

Non tutti i modelli devono essere file separati. Quando un oggetto è un'unità logica con più parti interattive interne, si usa un **singolo GLB con figli** (*children*). Nello scenario Manutenzione Elettromandrino, ad esempio:

- `pulpito.glb` — contiene il pannello di controllo con tutti i pulsanti (`Pulsante_mdi`, `chiave0`, `emergenza_off`, ecc.) come oggetti figli nella gerarchia GLB.
- `a500.glb` — contiene l'intera macchina con i sotto-assiemi mobili (`porta`, `naso`) animabili tramite `element = a500.porta`.

- `remote.glb` — il telecomando con i pulsanti figli (`pulsante_r_play`, `Pulsante_r_unlock`).

Aspetti tecnici

Per ottenere i nomi dei figli di un modello composito, carica la scena nel browser e usa la console di debug:

```
Scene3D.listChildNames(Scene3D.findModelByName('pulpito'))
```

Blender mostra la stessa gerarchia nel pannello *Outliner*.

[Immagine da inserire]

Screenshot del pannello Outliner di Blender con la gerarchia di `pulpito.glb` espansa: nodo radice `pulpito` con figli `Pulsante_mdi`, `Pulsante_tool`, `chiave0`, `chiave1`, `emergenza_off`, `pstart0`, `pstart1`.

Figura 2.4: L'Outliner di Blender mostra la gerarchia interna del modello `pulpito.glb`: ogni figlio diventa un elemento interattivo.

2.3 Origine, direzione e posizione iniziale

Ogni modello ha tre proprietà spaziali fondamentali che influenzano il comportamento nel tutorial.

2.3.1 Origine (pivot)

L'origine definita in Blender è il punto attorno al quale l'oggetto ruota e rispetto al quale vengono calcolate le traslazioni. Per gli oggetti che devono ruotare attorno a un proprio asse (viti, perni, alberi), il vincolo importante è che **l'origine giaccia sull'asse di rotazione**. Per una vite, qualunque punto lungo l'asse del gambo va bene: testa, centro o punta sono tutti equivalenti dal punto di vista del comportamento dell'animazione.

Quello che invece va evitato è un'origine *fuori asse* (spostata lateralmente rispetto al gambo): in quel caso la vite, durante `do : unscrew`, ruoterebbe descrivendo un cono invece di girare su sé stessa.

Trucchetto

In Blender usa *Object* → *Set Origin* → *Origin to 3D Cursor* dopo aver posizionato il cursore 3D sull'asse di rotazione dell'oggetto. La testa della vite è una scelta comoda perché coincide con il punto visivo di presa, ma non è un requisito tecnico: anche il centro del gambo o la punta funzionano allo stesso modo, purché stiano sull'asse.

[Immagine da inserire]

Confronto a tre viti identiche viste di lato. A sinistra origine spostata lateralmente rispetto al gambo (marcata con una X rossa, didascalia "fuori asse: sbagliato"), una freccia curva mostra una rotazione conica anomala. Al centro origine sul centro del gambo (punto verde, didascalia "sull'asse: giusto"). A destra origine sulla testa della vite (punto verde, didascalia "sull'asse: giusto"). Le due varianti di destra hanno entrambe una freccia di rotazione assiale corretta attorno al gambo.

Figura 2.5: L'origine deve stare sull'asse di rotazione del gambo. La posizione esatta lungo l'asse (testa, centro, punta) è indifferente; quello che conta è non spostarla lateralmente.

2.3.2 Direzione di movimento

Ogni modello ha una **direzione di movimento** associata nel file `scenes/homeconfig.ini`. Questa direzione viene usata da `unscrew`, `screw`, `extract` e `insert` per sapere lungo quale asse spostare l'oggetto:

```
[models/vite_coperchio_1.glb]
direction = -1, 0, 0    # svitamento verso X negativo

[models/filtro.glb]
direction = -1, 0, 0    # estrazione laterale verso X negativo

[models/vite_culatta_1.glb]
direction = 0, 0, 1     # svitamento verso Z positivo
```

Ricorda

La direzione è un vettore (x, y, z) che indica verso dove l'oggetto si muove quando viene estratto o svitato. Se non viene specificata, il programma non saprà in che direzione animare l'azione.

[Immagine da inserire]

Sistema di assi 3D ortogonali: X rosso (orizzontale), Y verde (verticale), Z blu (profondità). Una vite stilizzata posta nell'origine con freccia che esce lungo $-X$, etichettata "`direction = -1, 0, 0`".

Figura 2.6: Le tre direzioni dello spazio. Il vettore `direction` indica dove va l'oggetto quando lo sviti.

2.3.3 Posizione iniziale

La posizione iniziale di ogni modello viene definita in `scenes/homeconfig.ini`. Il sistema crea automaticamente un riferimento virtuale chiamato `nomemodello_original` che memorizza questa posizione di partenza. Questo riferimento torna utile:

- Per riportare un oggetto alla sua posizione iniziale dopo un movimento.
- Come punto di snap nel Drag & Drop (es. `snap targets filtro_original`).
- Per calcolare traslazioni relative con `do : move from filtro_original by (0,0,0.1)`.

Capitolo 3

Organizzare il progetto: cartelle e file

Una volta esportati i modelli 3D, prima di mettere mano al `tutorial.cvtscript` è utile fare il punto sulla struttura del progetto: dove vanno i file, come si dichiarano gli scenari, quali controlli effettuare. Sono passaggi rapidi ma fondamentali — senza una disposizione corretta dei file, il programma non riesce a trovare nulla.

Questo capitolo è una mini guida “operativa” che chiude la fase preparatoria. I file di configurazione qui introdotti (`homeconfig.ini`, `tool.ini`, `objects.ini`, `interfaceconfig.ini`) hanno una trattazione completa nel Capitolo 19; qui ne diamo solo la panoramica strettamente necessaria a far girare il tuo primo scenario.

3.1 Struttura delle cartelle del progetto

Le cartelle che contano per scrivere un tutorial sono organizzate in due livelli. Al **root del progetto** ci sono le cartelle “**globali**” condivise da tutti gli scenari (modelli 3D, video, immagini per display, icone della home), mentre dentro `scenes/` ci sono i **file di configurazione** e una **sottocartella per ogni scenario**.

```
campus_virtual_training/  # root del progetto
  models/                 # modelli 3D (.glb)  condivisi tra scenari
  media/                  # video e immagini per i message dei tutorial
  screens/                # frame per le animazioni dei display
  menuimages/             # immagini di anteprima della home page

  scenes/
    homeconfig.ini        # vetrina scenari (home page)
    interfaceconfig.ini   # personalizzazione mouse e camera

    Pompa_Becker/         # un sotto-cartella per ogni scenario
      tutorial.cvtscript   # sequenza guidata degli step
      tool.ini             # definizione degli utensili
      objects.ini          # stati e oggetti interattivi

    Manutenzione_Elettromandrino/
      tutorial.cvtscript
      tool.ini
```

objects.ini

Una volta sola, in alto

homeconfig.ini e interfaceconfig.ini si scrivono **una volta sola** dentro scenes/ e valgono per tutti gli scenari. Quando aggiungi un nuovo scenario, crei una nuova *sottocartella* dentro scenes/ con i suoi tre file (tutorial.cvtscript, tool.ini, objects.ini) e poi aggiungi una voce **[Nome scenario]** dentro homeconfig.ini.

Aspetti tecnici

Il percorso del modello nel tutorial è sempre relativo alla root del progetto. Scrivendo element = vite_coperchio_1, il sistema cerca automaticamente models/vite_coperchio_1.glb nella cartella globale al root del progetto. I video citati con video = stanno in media/, i frame delle animazioni in screens/.

[Immagine da inserire]

Albero ad icone della struttura del progetto su due livelli: al root le cartelle globali (models/ arancione, media/ viola, screens/ viola, menuimages/ viola) e la cartella scenes/. Dentro scenes/ i due file .ini (homeconfig.ini, interfaceconfig.ini) e le sottocartelle scenario (Pompa_Becker/, Manutenzione_Elettromandrino/), ognuna con esattamente tre file: tutorial.cvtscript, tool.ini, objects.ini.

Figura 3.1: La struttura completa: cartelle globali al root, configurazione e scenari dentro scenes/.

3.2 Registrare i modelli in homeconfig.ini

Affinché i modelli vengano caricati all'avvio dello scenario, devono essere elencati in scenes/homeconfig.ini. Per ogni modello servono due righe: il percorso e la direzione di uscita.

Estratto di homeconfig.ini

```
[Manutenzione pompa del vuoto]
CameraPos      = (-1.5, 0.5, -0.2)
CameraTarget    = (0, 0, 0)
tutorial        = scenes/Pompa_Becker/tutorial.cvtscript
tool            = scenes/Pompa_Becker/tool.ini

model          = models/corpo.glb
direction       = 0, 0, 1

model          = models/coperchio.glb
```

```
direction = -1, 0, 1

model      = models/filtro.glb
direction = -1, 0, 0

model      = models/vite_coperchio_1.glb
direction = -1, 0, 0
# ...altri modelli...
```

Ogni riga `model` = dichiara un file GLB da caricare; la riga `direction` = immediatamente successiva ne definisce la direzione di uscita. L'ordine di elencazione non influisce sul risultato finale.

Se dimentichi di registrare un modello in `scenes/homeconfig.ini`, il tutorial potrà riferirsi ad esso con `element` =, ma il modello non verrà caricato e lo step risulterà vuoto.

3.3 Checklist prima di iniziare a scrivere

Prima di passare alla scrittura del tutorial, verifica che ogni modello soddisfi questi requisiti:

- ☐ Il file è in formato `.glb`.
- ☐ Il nome è in minuscolo, senza spazi, descrittivo.
- ☐ L'origine è posizionata sull'asse di rotazione dell'oggetto (per le viti: lungo il gambo, non spostata lateralmente).
- ☐ Per i modelli composti, i figli hanno nomi univoci e riconoscibili nell'Outliner di Blender.
- ☐ Il modello è registrato in `scenes/homeconfig.ini` con la corretta `direction`.
- ☐ Il file è copiato nella cartella `models/` (o nella sottocartella dello scenario).

Trucchetto

Per verificare che tutti i modelli siano stati caricati correttamente, avvia lo scenario nel browser e usa il comando di debug `Scene3D.listAvailableObjects()` nella console: elencherà tutti gli oggetti presenti nella scena 3D.

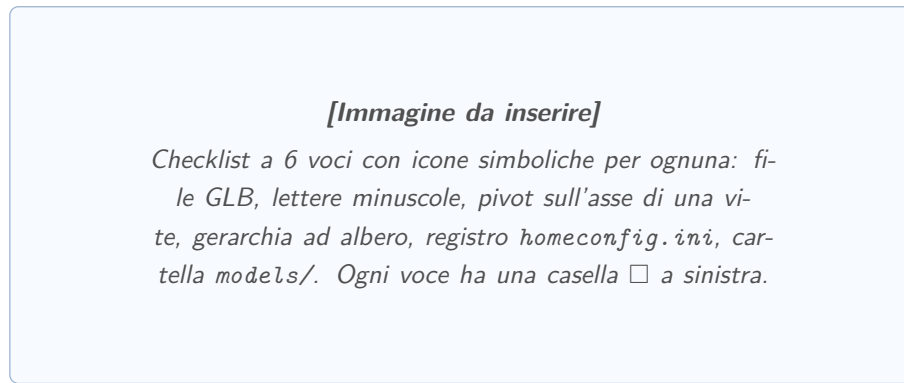


Figura 3.2: Sei punti da spuntare prima di passare alla scrittura del tutorial.

Capitolo 4

Come è organizzato un file cvtscript

Un file `tutorial.cvtscript` è composto da **blocchi**, ciascuno delimitato da un'intestazione tra parentesi quadre `[ ]`. È la stessa forma del classico *file INI* usato in molti file di configurazione, con due peculiarità: i blocchi sono organizzati in una **gerarchia a tre livelli** e all'interno di uno step accettano, oltre alle normali coppie `chiave = valore`, righe speciali introdotte da `do :` per descrivere le animazioni.

4.1 La forma di una riga

Dentro ogni blocco, le righe seguono sempre la stessa forma. Nessuna eccezione, nessun caso particolare:

```
| chiave = valore
```

[Immagine da inserire]

Zoom su una singola riga "chiave = valore" con tre etichette colorate sopra: "CHIAVE" (sopra chiave, blu), "UGUALE" (sopra =, arancione), "VALORE" (sopra valore, verde). Una freccia che evidenzia gli spazi attorno all'uguale.

Figura 4.1: Le tre parti di una riga: chiave, uguale, valore. Sempre la stessa struttura, ovunque nel file.

L'unica forma diversa che incontrerai è la riga d'azione `do : verbo ...`, trattata nel Capitolo 6: il separatore è `:` invece di `=` per evidenziare che si tratta di un *ordine*, non di una proprietà.

4.2 La gerarchia a tre livelli

Il file è organizzato come la struttura di un libro: **una cornice globale**, dentro la quale stanno **più capitoli**, ciascuno composto da **più passi**. I tre livelli si contengono uno dentro l'altro e corrispondono a tre tipi di blocco:

Livello	Blocco	Ruolo
1	<code>[scene]</code>	La cornice globale: condizioni iniziali della scena 3D, una sola volta all'inizio del file.
2	<code>[section "Titolo"]</code>	Un capitolo del tutorial. Un file ne può contenere uno o molti, in sequenza.
3	<code>[step "Titolo" flag]</code>	Il singolo passo che compone una sezione. Ogni sezione ne ha tipicamente da 5 a 30.

[Immagine da inserire]

Diagramma a tre livelli verticali: in alto un riquadro grande `[scene]` (cornice della stanza, con icona stanza vuota), al centro tre riquadri `[section "..."]` affiancati (capitoli del libro, con icona libro), in basso tanti riquadri piccoli `[step "..."]` (passi singoli, con icona orma). Frecce di contenimento tra i livelli.

Figura 4.2: I tre livelli della gerarchia: dalla cornice globale al singolo passo.

4.2.1 Livello 1 – `[scene]`: la cornice globale

`[scene]` è il blocco di apertura del file. Vi si dichiarano le condizioni iniziali condivise da tutto il tutorial:

- la posizione di partenza della telecamera (`camera = ...`);
- le posizioni iniziali dei modelli 3D che non si trovano già nel loro punto naturale (`position = ...`, `rotation = ...`).

Si scrive **una sola volta**, all'inizio del file:

```
[scene]
camera = position (-1.5, 0.5, -0.2), target (0, 0, 0), zoom 1.2, fade 2.5
```

Significa: “Telecamera, mettiti in quel punto, guarda verso il centro, zoomata 1.2, e quando ti sposti impiega 2.5 secondi”.

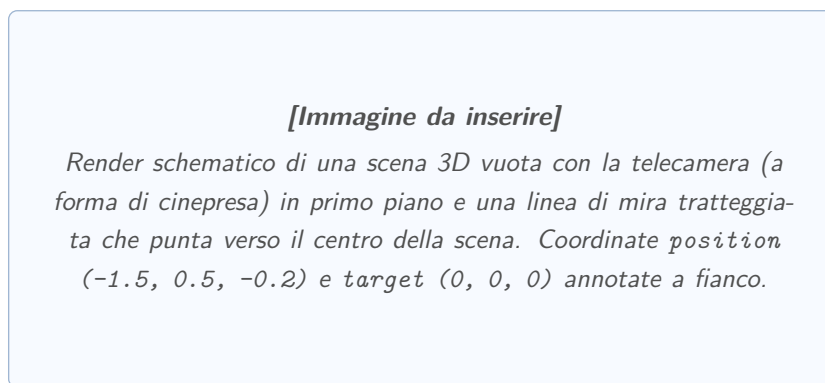


Figura 4.3: Il blocco `[scene]` fissa la telecamera all'avvio: posizione, target, zoom.

4.2.2 Livello 2 – `[section "Titolo"]`: il capitolo

Una `section` raggruppa una sequenza di step affini sotto un titolo descrittivo. Esempio: il file della pompa Becker contiene quattro sezioni che corrispondono a quattro tutorial distinti caricabili dalla home:

1. Pulizia Filtro
2. Riasssemblaggio
3. Sostituzione Palette – Smontaggio
4. Sostituzione Palette – Reverse

Ognuna si apre così:

```
[section "Pulizia Filtro"]
```

Il titolo va sempre fra **virgolette** " . . . " perché contiene spazi. Senza virgolette il parser interpreterebbe ogni parola del titolo come una bandierina e ti restituirebbe un errore.

4.2.3 Livello 3 – `[step "Titolo" flag]`: il singolo passo

Uno `step` è l'unità minima del tutorial: corrisponde a un singolo passaggio che lo studente deve compiere (svitare una vite, premere un pulsante, trascinare un filtro). Esempio:

```
[step "Prima vite" highlight]
```

L'intestazione contiene il **titolo** fra virgolette e, opzionalmente, una o più **bandierine** che ne modificano il comportamento (in questo caso `highlight` fa pulsare il fumetto della spiegazione). Le bandierine sono trattate nel Capitolo 11; se non te ne servono, ometti del tutto la parte dopo il titolo:

```
[step "Prima vite"]
```


I campi che si possono scrivere dentro lo step (l'`element`, l'eventuale `tool`, la `description`, le righe `do` :, ecc.) sono il cuore del linguaggio e occupano i prossimi capitoli, a partire dal Capitolo 5.

4.3 Altri blocchi accessori

Oltre ai tre livelli principali, CVTScript prevede alcuni blocchi **dichiarativi** per descrivere componenti riutilizzabili: `[state nome]` per le varianti visive di un oggetto (es. uno schermo digitale che mostra pagine diverse) e `[object nome]` per i modelli 3D con parti interattive (pulsanti del pulpito, leve, schermi).

Per igiene di progetto questi blocchi non si scrivono nel `tutorial.cvtscript`: vivono in un file separato chiamato `objects.ini` che il programma carica automaticamente prima del tutorial. Il Capitolo 18 ne descrive la sintassi; per ora basti sapere che *esistono* e che è grazie a loro che, dentro lo step, puoi scrivere semplicemente `element = pulpito.Pulsante_mdi` e il programma sa già di che pulsante si tratta.

Capitolo 5

Dentro uno step

[Immagine da inserire]

Blocco [step "Prima vite" highlight] esploso, con frecce colorate che escono da ogni riga (element, tool, description, do:, camera, ecc.) e puntano a etichette che spiegano il ruolo di ognuna in linguaggio naturale.

Figura 5.1: Tutte le righe di uno step: scegli quali ti servono di volta in volta.

Ogni step può contenere queste istruzioni (tutte in **minuscolo** con =):

Riga	Cosa fa	Esempio
<code>element =</code>	Il pezzo 3D dello step: vite, porta, pulsante, leva, qualsiasi cosa	<code>element = vite_coperchio_1</code> <code>element =</code> <code>pulpito.Pulsante_mdi</code>
<code>tool =</code>	Quale attrezzo serve allo studente	<code>tool = hand</code>
<code>description =</code>	Il testo del fumetto	<code>description = Rimuovi la</code> <code>prima vite</code>
<code>do :</code>	Un'azione (vedi capitolo 5)	<code>do : unscrew</code>
<code>camera =</code>	Dove guarda la telecamera	<code>camera = target filtro, zoom</code> <code>0.5</code>
<code>drag =</code>	Lo studente trascina con il mouse	<code>drag = filtro distance 0.3</code>
<code>after :</code>	Cosa succede dopo che lo studente clicca l' <code>element</code>	<code>after : schermo =</code> <code>schermo002</code>
<code>message =</code>	Mostra una finestra con un messaggio	<code>message = Attenzione, leggi</code> <code>qui!</code>

<code>title =</code>	Il titolo della finestra (con <code>message</code>)	<code>title = Leggi prima di continuare</code>
<code>video =</code>	Mostra un video nella finestra	<code>video = media/spiegazione.mp4</code>
<code>image =</code>	Mostra un'immagine nella finestra	<code>image = scenes/X/foto.jpg</code>
<code>hold =</code>	Lo studente prende/tiene/lascia un oggetto	<code>hold = pick remote at ...</code>
<code>view =</code>	Cambia la schermata di un display 3D	<code>view = main</code>
<code>position =</code>	Sposta un pezzo in un punto preciso	<code>position = filtro at (0, 0, 0)</code>
<code>rotation =</code>	Ruota un pezzo di tot gradi	<code>rotation = filtro by (0, 0, 90)</code>
<code>companion =</code>	Un pezzo che si muove insieme a quello principale	<code>companion = tubo follows master</code>
<code>animation =</code>	Mostra una sequenza di immagini animate 2D	<code>animation = folder x at (60%,60%) frame 20ms</code>

Scrivi le righe **tutte in minuscolo**. Niente `Element=`, niente `TOOL=`. Solo `element =` e `tool =`. È il modo per dire al programma: "Sto usando la sintassi nuova v3".

Capitolo 6

Gli ordini, l'istruzione `do` :

`do` : è l'istruzione più importante. Significa “**fai questa cosa adesso**”. La riga `do` : può apparire **più volte** in uno step. Le azioni vengono fatte **una dopo l'altra**, nell'ordine in cui le scrivi.

6.1 Quando parte un `do` : ?

Le azioni `do` : ... **non partono da sole all'inizio dello step**. Partono **quando lo studente clicca l'element** dello step. È l'interazione dello studente che mette in moto l'animazione.

In altre parole: ogni step ha un protagonista (l'`element`) e una serie di cose che gli devono accadere (i `do` :). Lo studente sceglie il momento: **cliccando sul protagonista** dà il via alla scena.

Esempio. Vediamo lo step della porta:

```
[step "Apri la porta"]
element = a500.porta
do      : rotate around (0.615, 0, 0) by (0, 90, 0) duration 1.0
```

Cosa succede dal punto di vista dello studente:

1. Lo step si carica. La porta (`a500.porta`) si **illumina di giallo** per dire “cliccami”.
2. Lo studente **clicca la porta**.
3. *Solo a quel punto* parte il `do` : `rotate` ...: la porta ruota di 90 gradi in 1 secondo.
4. Quando l'animazione è finita, lo step avanza al prossimo.

Senza il click dello studente, la porta resta lì ferma. Il programma aspetta.

6.1.1 Le tre eccezioni: `auto`, `machine`, e l'attrezzo

Ci sono tre situazioni in cui il `do` : parte **senza un click diretto** sull'`element`:

- **Step con bandierina `auto`:** lo studente **ha già equipaggiato il tool** dichiarato e il programma simula il click in automatico. Si usa per le viti consecutive: la prima la fa lo studente, le altre sono in cascata.
- **Step con bandierina `machine`:** la macchina si muove da sola, senza nemmeno il tool. Esempio: il carro dell'asse Y che si sposta automaticamente durante un cambio utensile.
- **Step con `tool = ... + do ::`** lo studente **equipaggia l'attrezzo dalla cassetta** e poi clicca l'`element` con quell'attrezzo. È sempre un click sull'`element`, solo mediato dal tool.

[Immagine da inserire]

Quattro vignette in fila. (1) Cursore che scende verso un'`element` illuminata di giallo → click → `do` che parte. (2) Lo stesso ma il cursore ha forma dell'attrezzo (chiave a brugola). (3) Step con la bandierina `auto`: il cursore non c'è, una freccia "automatico" avvia il `do`. (4) Step `machine`: la macchina che si muove da sola, etichetta "nessun click necessario".

Figura 6.1: Quando parte un `do ::` nei tre casi normali al click dello studente, nei due automatici (`auto/machine`) da solo.

6.1.2 Più `do` : in fila = una sola animazione

Se metti più righe `do` : nello stesso step, vengono eseguite **una dopo l'altra** a partire dal singolo click dello studente. Lo studente clicca **una volta**, e parte tutta la sequenza.

```
[step "Estrai la flangia" highlight]
element      = flangia
description = Stacca la flangia dalla macchina.
do           : move by (0, 0, 0.5) duration 0.5
do           : rotate by (90, 0, 0) duration 0.5
do           : place duration 0.2
```

Cosa vede lo studente. Clicca la flangia una sola volta. Poi: prima la flangia si sposta lungo Z (mezzo secondo); poi ruota di 90 gradi (altro mezzo secondo); poi si appoggia a terra (un quinto di secondo). Tre animazioni concatenate, un solo click.

6.2 Il vocabolario degli ordini

[Immagine da inserire]

Tabella visiva 3×3 con le 9 azioni: ognuna ha un'icona simbolica e il nome inglese sotto. *unscrew* (vite con frecce circolari + uscita), *screw* (vite che entra), *extract* (freccia in linea), *insert* (freccia inversa), *place* (oggetto che cade), *move* (cubo con vettore), *rotate* (cerchio rotatorio), *pump* (oscillazione ↔), *idle* (cerchio reset).

Figura 6.2: Il vocabolario completo: nove ordini per descrivere ogni movimento.

Sono solo **nove parole**:

Ordine	Cosa fa	Esempio
unscrew	Svita la vite (rotazione + estrazione)	do : unscrew distance 0.3
screw	Avvita la vite (inverso di unscrew)	do : screw distance 0.005
extract	Estrae il pezzo lungo l'asse	do : extract distance 0.4
insert	Inserisce il pezzo	do : insert distance 0.4
place	Appoggia a terra il pezzo	do : place duration 0.5
move	Sposta il pezzo (traslazione)	do : move by (0,0,-0.1) duration 0.2
rotate	Ruota il pezzo	do : rotate by (0,0,-90) duration 0.8
pump	Pompa avanti-indietro (cicli)	do : pump along x amplitude 0.08...
idle	Operazioni speciali	do : idle reset

6.3 Spiegazioni semplici di ognuno

6.3.1 unscrew – Svita la vite

```
do : unscrew                # Svita di mezzo metro (default)
do : unscrew distance 0.3   # Svita di 30 centimetri
```

Pensa a una vite: per svitarla devi **gitarla tante volte** e **tirla fuori**. **unscrew** fa entrambe le cose insieme.

Nota

La direzione (su, giù, verso destra) è scritta nel file `scenes/homeconfig.ini` per ogni modello. Tu non te ne devi preoccupare.

[Immagine da inserire]

Tre fotogrammi in sequenza orizzontale: vite avvitata nel pezzo (frame 1), vite a metà strada con freccia di rotazione e freccia di traslazione (frame 2), vite completamente fuori e separata (frame 3). Frecce di flusso tra i frame.

Figura 6.3: **unscrew** combina rotazione e traslazione lungo l'asse di svitatura.

6.3.2 screw – Avvita la vite

L'opposto di **unscrew**. La vite **rientra** nel suo buco.

```
do : screw distance 0.02
```

6.3.3 extract / insert – Estrae / Inserisce

Come svitare ma **senza la rotazione**. Solo lo spostamento.

```
do : extract distance 0.6      # tira fuori di 60 cm
do : insert distance 0.6      # rimette dentro
```

[Immagine da inserire]

*Pezzo cilindrico (es. filtro) che esce in linea retta dalla sua sede, senza rotazione. Sopra l'asse di estrazione una doppia freccia: → etichettata *extract*, ← etichettata *insert*. Il pezzo non ruota mai.*

Figura 6.4: **extract** è solo traslazione: niente rotazione, solo movimento lungo la direzione del modello.

6.3.4 place – Appoggia a terra

Il pezzo **scende e si appoggia per terra**. Utile dopo che lo hai svitato e tolto.

```
do : place duration 0.5      # impiega mezzo secondo a scendere
```

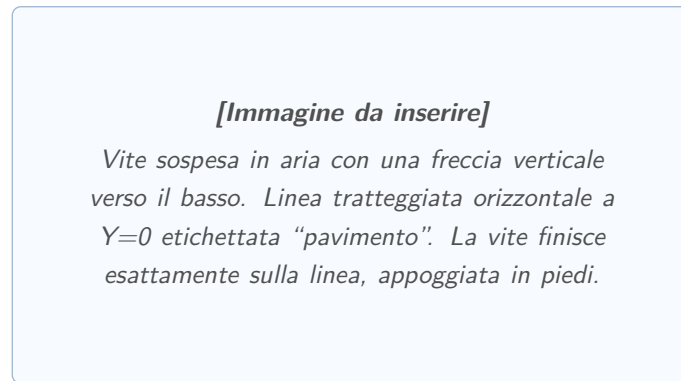


Figura 6.5: `place` calcola il pavimento e fa scendere il pezzo dolcemente fino a $Y=0$.

6.3.5 move – Sposta in linea retta

```
do : move by (1, 0, 0) duration 1.0
```

Significa: sposta il pezzo di 1 metro lungo l'asse X, in 1 secondo.

I tre numeri tra parentesi sono **(X, Y, Z)** – esattamente come tre direzioni:

- **X** = destra/sinistra (numeri positivi = destra)
- **Y** = su/giù (positivi = su)
- **Z** = avanti/indietro (positivi = avanti)

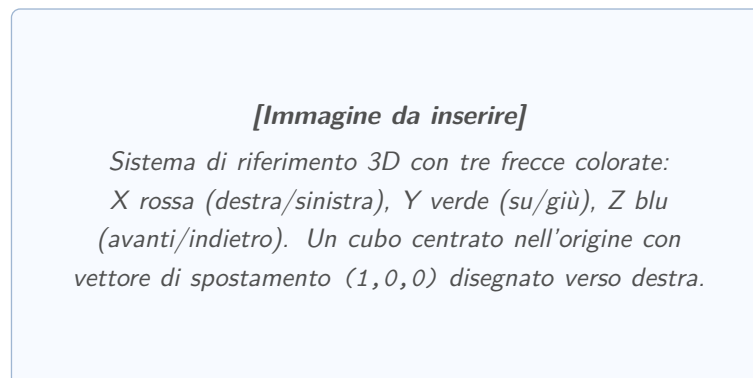


Figura 6.6: Le tre coordinate (X, Y, Z) come tre direzioni dello spazio.

Versione speciale che parte da un altro pezzo:

```
do : move from tappino_grasso_dx_original by (-0.2, 0, 0) duration 0.5
```

Significa: "Vai vicino al tappino di destra (alla sua posizione originale), poi spostati di -0.2 lungo X". Si usa quando vuoi posizionare un oggetto **vicino a un altro**.

6.3.6 rotate – Gira

```
do : rotate by (0, 0, 90) duration 1.0
```


Significa: ruota di 90 gradi attorno all'asse Z, in 1 secondo.

Versione speciale “ruota intorno a un punto preciso”:

```
do : rotate around (0, 0.1, 0) by (0, 0, -90) duration 0.8
```

Significa: ruota di -90 gradi sull'asse Z, ma il **perno della rotazione** è il punto (0, 0.1, 0). È come quando una porta gira intorno alla cerniera.

[Immagine da inserire]

Confronto a due immagini: a sinistra cubo che ruota attorno al suo centro (rotate by, perno = origine pezzo). A destra cubo che ruota attorno a un punto esterno marcato con un cardine (rotate around, come una porta che gira sui cardini).

Figura 6.7: La differenza tra rotate by (perno = origine pezzo) e rotate around (perno = punto esterno).

6.3.7 pump – Pompare

Il pezzo si muove **avanti e indietro** tante volte (es. una pompetta del grasso).

```
do : pump along x amplitude 0.08 cycles 10 duration 0.1 from  
tappino_grasso_dx_original
```

Si legge: “Pompa lungo X, ampiezza 8 cm, fai 10 cicli, ogni ciclo dura 0.1 secondi, partendo dal tappino di destra”.

Versione asimmetrica (il “su” e il “giù” non sono uguali):

```
do : pump from tappino_grasso_dx_original between (-0.08, 0, 0) and  
(-0.09, 0, 0) cycles 10 duration 0.1
```

[Immagine da inserire]

Pompette del grasso con stantuffo che si muove avanti-indietro: doppia freccia ↔ etichettata “ampiezza”. Sotto, una linea ondulata sinusoidale con N picchi etichettati “cicli=10”. A fianco una clessidra con scritto “durata=0.1s per ciclo”.

Figura 6.8: **pump** ripete avanti-indietro N volte: utile per simulare pompette del grasso.

6.3.8 `idle reset` – Resetta tutto

```
| do : idle reset
```

Mette tutti i pezzi nella loro posizione originale. Si usa solitamente all'inizio di una **section** per “ripulire” la scena.

Capitolo 7

Gli strumenti: i vari attrezzi

Lo studente, durante il tutorial, sceglie un attrezzo. Tu gli dici quale serve con la riga `tool = ....`. Sono **poche**, parole inglesi semplici:

Riga nel file	Attrezzo	Quando usarlo
<code>tool = hand</code>	Mano	Per pezzi che si tolgono a mano (tappi, coperchi)
<code>tool = hex_key</code>	Chiave a brugola	Per viti a brugola (esagonali)
<code>tool = wrench</code>	Chiave inglese	Per dadi e bulloni
<code>tool = air</code>	Aria compressa	Per soffiare via la sporcizia
<code>tool = spray</code>	Spray lubrificante	Per ingrassare

[Immagine da inserire]

*Cassetta degli attrezzi aperta con cinque scomparti, ognuno con icona e doppia etichetta (italiana sopra, inglese sotto): mano (*hand*), brugola (*hex_key*), chiave inglese (*wrench*), pistola aria compressa (*air*), bomboletta spray (*spray*).*

Figura 7.1: Gli attrezzi disponibili: i nomi inglesi sono quelli da usare nella riga `tool =`.

Esempio completo:

```
[step "Rimuovi la vite" highlight]
element      = vite_culatta_1
tool         = hex_key
description  = Usa la chiave a brugola da 6mm per svitare la prima vite.
do           : unscrew distance 0.3
do           : place duration 0.2
```

[Immagine da inserire]

Screenshot della barra degli strumenti reale dell'app: 4 slot in basso allo schermo, ognuno con icona dell'attrezzo e label sotto. Uno slot è evidenziato (selezionato) con bordo colorato. Annotazioni numerate 1–4 sopra ogni slot.

Figura 7.2: La cassetta degli attrezzi nell'interfaccia: lo studente sceglie da qui prima di ogni azione.

[Immagine da inserire]

Galleria di 5 cursori: per ognuno (brugola, chiave inglese, mano, aria, spray) due frame affiancati: "normale" a sinistra e "premuto" a destra. Etichetta sotto ogni coppia.

Figura 7.3: I cursori dei cinque attrezzi: cambiano forma quando il mouse è premuto.

Capitolo 8

La camera: dove guarda lo studente

La telecamera è gli **occhi dello studente**. Tu decidi dove guardare con la riga `camera = ....`. Si scrive **tutto su una riga**, separando con virgole:

```
| camera = position (1, 2, 3), target (0, 0, 0), zoom 1.5, fade 2.0
```

8.1 Le parti della camera

[Immagine da inserire]

Schema 3D della telecamera: una cinepresa stilizzata in posizione position, vettore tratteggiato verso target, cono giallo del campo visivo (FOV), e cerchio orizzontale che rappresenta il pivot. Misurato anche il distance come segmento tra position e target.

Figura 8.1: Le proprietà geometriche della camera: ogni numero ha un significato preciso nello spazio.

Pezzettino	Significato
position (x, y, z)	Dove sta la telecamera nello spazio
target (x, y, z)	Dove sta guardando
target nome_pezzo	Punta a un pezzo (senza coordinate)
zoom 1.5	Quanto è zoomata (più alto = più vicino)
fade 2.0	Quanti secondi impiega per spostarsi
pivot (x, y, z)	Il punto attorno a cui ruota
distance 0.5	Distanza dal target
fov 75	“Angolo di vista” della camera
rotation (rx, ry, rz)	Rotazione della camera

`free within (-3.14, 3.14)` Lo studente può girare liberamente

[Immagine da inserire]

Stessa scena vista con due valori di fov: a sinistra fov 45 (campo stretto, effetto teleobiettivo, soggetto grande), a destra fov 90 (campo ampio, effetto grandangolo, si vede di più). Cono di vista sovrapposto.

Figura 8.2: Più alto è il FOV, più ampio è il campo di vista (effetto grandangolo).

[Immagine da inserire]

Emisfero (cupola) attorno a un oggetto al centro: archi colorati in verde indicano l'intervallo di rotazione consentito da `free within (-3.14, 3.14)`, archi rossi al di fuori indicano i limiti vietati. Una telecamera si muove lungo l'arco verde.

Figura 8.3: I limiti `free within` impediscono allo studente di disorientarsi girando troppo la camera.

Trucchetto

Nelle **section** e negli **step** puoi scrivere **solo i pezzi che cambiano**. Il programma ricorda gli altri dal contesto precedente.

Esempio: se uno step è quasi uguale al precedente ma cambi solo il target:

```
[step "Tappino Sinistro" highlight]
camera      = target tappino_grasso_sx
description = Monta il tappino di sinistra.
```

Capitolo 9

Trascinare gli oggetti (drag & drop)

A volte vuoi che sia lo **studente a trascinare** un pezzo nella sua sede (non automatico). Si fa con la riga `drag = ...`:

```
| drag = filtro distance 0.3
```

Significa: “Lo studente può trascinare il filtro. Quando si avvicina entro 30 cm dal punto di destinazione, il pezzo **si aggancia automaticamente**”.

[Immagine da inserire]

Tre fotogrammi orizzontali: 1) filtro lontano dalla sua sede, cursore mano che lo trascina; 2) filtro entro la sfera tratteggiata di soglia (distance 0.3), il bordo della sfera è visibile; 3) filtro agganciato perfettamente nella sede, indicatore di snap acceso (verde).

Figura 9.1: La meccanica dello snap: quando l'oggetto entra nella soglia di distanza, si aggancia automaticamente.

9.1 Più oggetti contemporaneamente

```
| drag = vite_1, vite_2, vite_3, vite_4 distance 0.2
```

Significa: “Lo studente può trascinare le 4 viti, in qualsiasi ordine”.

9.2 Punto di aggancio personalizzato

Forma	Cosa fa
<code>snap offset (0, 0, 0.01)</code>	Si aggancia un po' sollevato (1 cm sopra)

<code>snap pivot (x, y, z)</code>	Si aggancia con il pivot in quel punto
<code>snap targets a, b, c</code>	Si aggancia in uno qualsiasi di questi punti

Esempio con tutto:

```
drag = filtro distance 1.5 snap offset (0, 0, 0.01)
```

```
drag = vite distance 1.5 snap targets foro_1.origin, foro_2.origin
```

[Immagine da inserire]

Tre riquadri affiancati con esempi di snap: 1) `snap offset (0,0,0.01)` — pezzo agganciato 1 cm sopra la sede (linea tratteggiata di offset visibile); 2) `snap pivot (x,y,z)` — pezzo con un perno colorato spostato dal centro; 3) `snap targets a, b, c` — vite con 3 fori candidati (bersagli) di cui quello più vicino si illumina.

Figura 9.2: Tre modi di definire dove un oggetto si aggancia: offset, pivot custom, lista di target.

Capitolo 10

Pulsanti, porte, leve: tutto è element

[Immagine da inserire]

Render del pulpito 3D dell'Elettromandrino con callout numerati che evidenziano i singoli pulsanti: Pulsante_mdi, Pulsante_tool, chiave0, chiave1, emergenza_off, pstart0, pstart1. Sopra il modello l'etichetta "pulpito.glb".

Figura 10.1: Il pulpito di una macchina utensile: ogni pulsante è un figlio del modello `pulpito.glb`.

Una delle cose più importanti del CVTScript v3 è una piccola idea che cambia tutto: **tutto quello su cui lo studente clicca si chiama `element`**. Una vite, un pulsante del pulpito, una porta, una leva, una chiave da girare: una sola parola, `element`, per dire al programma “questo è il pezzo di questo step”.

Il programma capisce **da solo** cosa deve succedere quando lo studente clicca un `element`, guardando le *altre righe* dello step: se c'è `do` : parte un'animazione, se c'è `after` : cambia uno stato, se c'è `tool` = si usa un attrezzo, se c'è la bandierina `auto` non serve neanche cliccare. **Tu scrivi solo cosa vuoi che succeda**: il programma collega da sé i pezzi.

Perché “element” e basta

Nelle prime versioni del v3 c'era una parola `button` = per i pulsanti 3D. Ma una porta non è un pulsante. Una leva non è un pulsante. Una vite non è un pulsante. Chiamarli tutti “button” confondeva chi leggeva il file. Allora abbiamo unificato: **tutto è `element`**, e il contesto chiarisce di cosa si tratta. La parola `button` = continua a funzionare per non rompere i tutorial vecchi, ma il programma ti avvisa che è deprecata.

10.1 Caso A – Click su un pulsante che cambia uno schermo

Quando lo studente deve **premere un pulsante 3D** (es. il pulsante MDI sul pulpito della macchina), scrivi semplicemente l'`element` e il `after` ::

```
[step "Vai alla schermata MDI" highlight]
description = Premi il pulsante MDI sul pulpito.
element     = pulpito.Pulsante_mdi
after       : schermo = schermo002
```

Si legge:

- “Il pezzo dello step è il pulsante chiamato Pulsante_mdi dentro l'oggetto pulpito”.
- “Quando lo studente lo clicca, cambia la variante dello schermo a schermo002”.
- Niente `do` : significa che non c'è animazione da aspettare: si passa subito al prossimo step.

10.2 Caso B – Click su una porta che si apre

Una porta non è un pulsante: è una porta. Ma per il programma è anch'essa un `element`. La differenza è che ha un'animazione (`do` :) invece di un `after` ::

```
[step "Apri la porta"]
description = Apri la porta della cabina.
element     = a500.porta
do          : rotate around (0.615, 0, 0) by (0, 90, 0) duration 1.0
```

Si legge:

- “Il pezzo dello step è la porta della macchina (a500.porta)”.
- “Quando lo studente la clicca, la porta ruota di 90 gradi attorno al punto (0.615, 0, 0) in 1 secondo”.
- Quando l'animazione è finita, lo step avanza da solo.

10.3 Caso C – Click su una vite con un attrezzo

Quando l'`element` richiede un attrezzo (chiave a brugola, chiave inglese, spray...) aggiungi `tool = ....`. Lo studente prima equipaggia l'attrezzo, poi clicca l'`element`:

```
[step "Rimuovi vite culatta"]
element     = vite_culatta_1
tool        = hex_key
description = Svita la vite con la chiave a brugola.
do          : unscrew distance 0.3
```

10.4 after con effetti multipli

Se cliccare l'`element` deve cambiare **più cose** contemporaneamente, separa con `;`:

```
element = pulpito.pstart0
after : start = pstart1; schermo = schermo002
```

Significa: “Quando lo studente clicca il pulsante di accensione, cambia sia la variante del pulsante stesso (acceso) sia quella dello schermo”.

after funziona anche senza click

`after` : significa “**alla fine di questo step**”. Se lo step ha un `element` = cliccabile, “alla fine” = “dopo il click”. Se invece lo step è `auto` o `machine` (senza click), “alla fine” = “quando lo step parte”.

Esempio – step `machine` che cambia variante senza click:

```
[step "Scambio Utensile" machine]
after : tool = tool0
```

Lo studente non deve cliccare niente: la macchina cambia da sola la variante del tool e va al prossimo step.

10.5 La tabella che decide tutto

Ecco la regola completa. Il programma guarda quali righe ci sono nello step e sceglie cosa far succedere al click:

tool?	do :?	after :?	flag auto/machine?	Cosa succede al click
sì	sì	—	no	Lo studente equipaggia il tool, clicca l' <code>element</code> con il tool, parte l'animazione.
no	sì	—	no	Lo studente clicca l' <code>element</code> , parte l'animazione.
qualunque	no	sì	no	Lo studente clicca l' <code>element</code> , partono gli effetti di <code>after</code> :, lo step avanza.
qualunque	no	no	no	Lo studente clicca l' <code>element</code> , lo step avanza.
qualunque	qualunque	qualunque	sì	Non serve cliccare: parte tutto da solo. L' <code>element</code> è solo il bersaglio dell'animazione.

[Immagine da inserire]

Quattro mini-scene affiancate che condividono lo stesso schema di partenza “il programma guarda lo step”. Scena 1: vite + attrezzo → “flusso tool”. Scena 2: porta che ruota → “click + animazione”. Scena 3: pulsante che cambia schermo → “click + after”. Scena 4: ingranaggio rotante → “machine, automatico”. Sopra le scene un'unica scritta: *element = X*.

Figura 10.2: Una sola keyword, quattro comportamenti: il programma deduce dal contesto.

10.6 Click ripetuti: repeat N

A volte vuoi che lo studente clicchi lo stesso *element* **più volte** (per esempio per simulare “apri e chiudi la pinza 8 volte e verifica che funzioni”). Aggiungi *repeat N* al valore dell'*element*:

```
[step "Apri e chiudi la pinza" highlight]
element    = remote.Pulsante_r_unlock repeat 8
animation  = folder screens/mandrino at (60%, 60%) frame 20ms
```

Significa: “Lo studente deve premere il pulsante **8 volte**. Ogni volta appare un'**immagine diversa** dalla cartella screens/mandrino (animazione a frame, 20 millisecondi tra una e l'altra). Dopo l'ottavo click, lo step avanza”.

[Immagine da inserire]

Pulsante 3D al centro con contatore “+1 +1 +1” che sale (1/8, 2/8...8/8). Sopra, una sequenza di mini-schermate che cambiano a ogni click (filmstrip). All'ottavo click, il fumetto si chiude e parte la freccia di avanzamento.

Figura 10.3: repeat N accoppiato a animation: ogni click avanza un frame, all'ennesimo lo step finisce.

Capitolo 11

Le bandierine auto, machine, highlight

[Immagine da inserire]

*Tre bandierine stilizzate affiancate, ognuna con icona simbolica e nome: **auto** (ingranaggio in movimento), **machine** (silhouette di macchina utensile), **highlight** (lampada accesa con onde di luce). Sotto ogni bandierina una frase breve "chi muove cosa, chi attira l'attenzione".*

Figura 11.1: Le tre bandierine: chi muove cosa, e chi attira l'attenzione dello studente.

Sono **etichette speciali** che si scrivono **dopo il nome dello step**, nella parentesi quadra:

```
[step "Nome dello step" auto, highlight]
```

Bandierina	Significato
auto	Lo step si esegue da solo : equipaggia l'attrezzo, fa le azioni, e va al prossimo step. Lo studente non deve cliccare.
machine	Come auto ma senza attrezzo . Si usa per movimenti automatici della macchina (es. "il carro Y si sposta").
highlight	Il fumetto della spiegazione lampeggia per attirare l'attenzione. Si usa per i passi importanti dove vuoi che lo studente legga.

Si possono **combinare** separandole con virgola:

```
[step "Apertura magazzino" highlight, machine]
```

Significa: il fumetto lampeggia E lo step si esegue automaticamente.

[Immagine da inserire]

*Due screenshot affiancati della stessa scena: a sinistra fumetto normale (testo statico, bordo sottile); a destra fumetto in stato **highlight** (bordo luminoso, alone giallo-arancio attorno al riquadro, frecce di pulsazione). Etichetta sotto: "normale" / "highlight".*

Figura 11.2: L'effetto **highlight**: il fumetto pulsa per dirigere l'attenzione dello studente al testo importante.

Capitolo 12

Tenere oggetti in mano (hold)

A volte lo studente deve **prendere un oggetto in mano** (per esempio un telecomando o una pistola di aria compressa) e poi rimetterlo giù. Si gestisce con la riga `hold = ...`, che ha tre forme.

[Immagine da inserire]

Sequenza a tre stati orizzontali: 1) telecomando posato sul tavolo (etichetta `pick`), 2) telecomando sospeso davanti alla telecamera in primo piano (etichetta `held`), 3) telecomando di nuovo sul tavolo (etichetta `release`). Freccie di transizione tra gli stati con i nomi delle azioni.

Figura 12.1: Il ciclo di vita di un oggetto in mano: `pick` → `held` → `release`.

12.1 `hold = pick` – Prendi in mano un oggetto

```
hold = pick remote at (-0.25, -0.07, 0.25) facing (0, 5, 0)
```

Si legge: “**Prendi** l’oggetto chiamato `remote`, mettilo nella **posizione** `(-0.25, -0.07, 0.25)` rispetto alla telecamera, con la **rotazione** `(0, 5, 0)` (in gradi)”.

L’oggetto seguirà la telecamera come se fosse tenuto davanti agli occhi. Le parti `at (...)` e `facing (...)` sono **opzionali**: se non le metti il sistema usa una posizione di default.

[Immagine da inserire]

Telecamera in primo piano (cinepresa stilizzata) con il telecomando sospeso davanti agli occhi. Vettore at $(-0.25, -0.07, 0.25)$ disegnato dalla camera al telecomando. Triedro di rotazione $facing$ $(0, 5, 0)$ che mostra l'inclinazione dell'oggetto (gradi).

Figura 12.2: at posiziona l'oggetto rispetto alla camera; $facing$ ne imposta l'orientamento.

12.2 $hold = held$ – Lo step richiede l'oggetto già in mano

```
hold = held remote
```

Significa: “Questo step parte solo se l'oggetto $remote$ è **già in mano** dal passo precedente”. Si usa quando hai una sequenza di step in cui l'oggetto viene preso una volta sola e poi usato in più passi.

12.3 $hold = release$ – Rimetti giù

```
hold = release
```

L'oggetto torna nella sua posizione originale nella scena. Lo studente **lo molla**.

Sequenza completa: prendi, usa, lascia

```
[step "Prendi il telecomando" highlight]
description = Prendi il telecomando dal tavolo.
tool        = hand
hold        = pick remote at (-0.25, -0.07, 0.25) facing (0, 5, 0)
view        = main

[step "Premi Start sul telecomando"]
description = Premi il pulsante Start.
hold        = held remote
element     = remote.pulsante_r_play
after       : schermo = schermo004

[step "Rimetti giù il telecomando"]
description = Rimetti il telecomando al suo posto.
hold        = release
```


Capitolo 13

Mostrare un messaggio sullo schermo

A volte vuoi mostrare allo studente una **finestra di avviso** con un testo, magari un'**immagine** o un **video** di spiegazione. Si usa `message` = accompagnato (in modo opzionale) da `title` =, `image` = e `video` =.

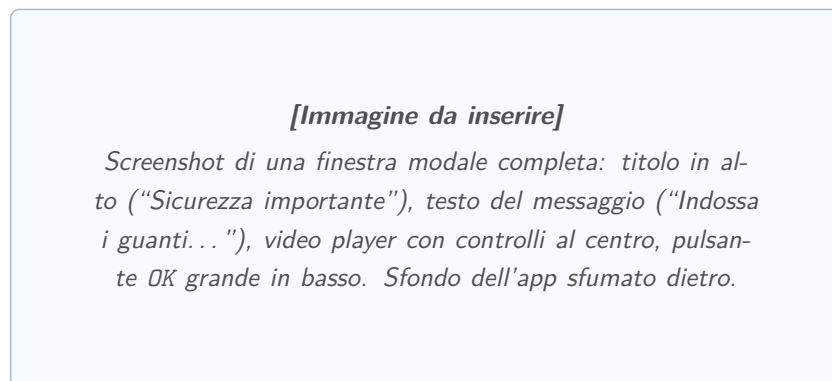


Figura 13.1: Una finestra modale completa: titolo, messaggio, video opzionale, OK per chiudere.

```
[step "Attenzione lubrificante" highlight]
message = Indossa i guanti prima di toccare il grasso.
title    = Sicurezza importante
video    = media/lubelett.mp4
```

Cosa succede: appare al centro dello schermo una finestra grande con:

- Il **titolo** "Sicurezza importante".
- Il **testo** "Indossa i guanti...".
- Il **video** che parte automaticamente con i controlli (play/pause).
- Un pulsante **OK** per chiudere e andare avanti.

Combinazioni possibili

- `message = da solo` → finestra con solo testo
- `message = + title =` → testo con titolo personalizzato
- `message = + image =` → testo + immagine
- `message = + video =` → testo + video (con player)

Se non scrivi `title =`, il titolo della finestra sarà uguale al nome dello step.

[Immagine da inserire]

Galleria di 4 mock-up di finestra modale, in fila: 1) solo testo (no titolo no media), 2) testo + titolo personalizzato, 3) testo + immagine al centro, 4) testo + video con controlli player. Etichetta sotto ognuna.

Figura 13.2: Le quattro combinazioni di `message`: scegli ciò che ti serve.

Trucchetto

Puoi mettere `message = insieme` ad altre azioni dello step (es. con `drag =` o `do :`). La finestra appare **prima**, lo studente la chiude con OK, poi parte il resto.

Capitolo 14

Mettere a posto i pezzi prima di iniziare

Quando carichi la scena 3D, ogni pezzo si trova nella sua posizione “naturale” (quella in cui è stato modellato in Blender). Ma a volte tu vuoi che la scena **parta in un certo modo specifico**: per esempio in un tutorial di rimontaggio, vuoi che le viti siano già **tolte e appoggiate per terra**, in modo che lo studente le trascini al loro posto.

Per questo ci sono `position =` e `rotation =`.

14.1 `position` – Mettere un pezzo in un punto

```
| position = vite_coperchio_1 at (-0.500, -0.349, 0.000)
```

Si legge: “Metti la vite_coperchio_1 nel punto $(-0.5, -0.35, 0)$ rispetto al centro della scena”.

14.2 `rotation` – Ruotare un pezzo

```
| rotation = coperchio by (0, 0, -90)
```

Si legge: “Ruota il coperchio di -90 gradi sull’asse Z”.

14.3 Dove si scrivono?

- Nella `[scene]`: si applicano **una sola volta** all’inizio del tutorial.
- In una `[section "..."]`: si applicano all’inizio del capitolo, prima del primo step.
- In uno `[step "..."]`: si applicano **quando lo step parte**.

Posizionare i pezzi smontati per il rimontaggio

```
| [section "Riassemblaggio"]  
camera = position (-1.5, 0.5, -0.2), target (0, 0, 0), zoom 1.2
```

```
# Pezzi che partono smontati e per terra
position = coperchio at (-1.148, -0.261, 0.500)
rotation = coperchio by (0, 0, -90)
position = filtro at (-0.300, -0.142, 0.500)
position = vite_coperchio_1 at (-0.500, -0.349, 0)
position = vite_coperchio_2 at (-0.500, -0.349, 0)

[step "Tappino destro" highlight]
description = Monta il tappino di destra.
tool        = hand
drag        = tappino_grasso_dx distance 0.2
```

[Immagine da inserire]

Confronto a due render della stessa scena: a sinistra "Default" — pompa intera già montata; a destra "Dopo position/rotation" — coperchio per terra ruotato di 90°, filtro a fianco, viti sparpagliate sul pavimento. Etichetta sopra: "Stato iniziale per il rimontaggio".

Figura 14.1: La stessa scena prima e dopo i comandi position/rotation della sezione di rimontaggio.

Capitolo 15

Pezzi che si muovono insieme (companion)

A volte un pezzo si muove e **ne tira un altro**. Per esempio: quando tiri fuori la flangia della pompa, il **tubo del grasso** attaccato dietro viene tirato anche lui.

In questi casi il pezzo principale (che lo studente sposta) si chiama **master**. I pezzi che lo seguono si chiamano **compagni** (**companion**).

15.1 companion = X follows master – Mi attacco al master

```
[step "Allontana la flangia"]
element    = flangia
tool       = hand
companion  = tubograsso follows master
do         : move by (0, 0, 0.2) duration 0.5
```

Significa: “Quando la **flangia** (master) si sposta di 20 cm avanti, il **tubograsso** la **segue** attaccato”.

[Immagine da inserire]

Flangia (master, blu) con catena/corda visuale che la lega al tubograsso (companion, arancione). Una freccia di movimento parte dalla flangia, e il tubograsso si muove rigidamente nella stessa direzione e quantità. Etichetta “master” sulla flangia, “companion” sul tubo.

Figura 15.1: **follows master**: il companion è agganciato e segue rigidamente il movimento del master.

15.2 companion = X moves by (...) duration t – Movimento separato

```
companion = flangia moves by (0, 0, 0.005) duration 0.5
companion = tubogrosso moves by (0, 0, 0.005) duration 0.5
```

In questo caso il companion fa un **movimento suo**, in parallelo al master. Si usa per esempio quando si avvitano viti estrattori e **tutta la flangia avanza un pochino** a ogni giro di vite.

[Immagine da inserire]

Master e companion disegnati come due oggetti distinti, ognuno con la propria freccia di movimento parallela ma indipendente. Una clessidra centrale unisce le due frecce per indicare "stessa durata". Nessuna catena tra i due.

Figura 15.2: **moves by**: il companion fa il proprio movimento in parallelo, non agganciato al master.

Nota

Si possono mettere **più righe** **companion = ...** nello stesso step: ogni pezzo segue/si muove come scritto nella sua riga.

Capitolo 16

Schermate di un display 3D (view)

[Immagine da inserire]

Quattro pannelli affiancati di uno stesso display 3D: "home" (icone delle app), "mdi" (campo di input), "tool" (parametri utensile), "running" (barra avanzamento). Ogni vista ha una etichetta sopra con il nome view.

Figura 16.1: Le viste di un display digitale: ogni view è un GLB diverso che si scambia in tempo reale.

Alcuni oggetti della scena hanno uno **schermo digitale** (per esempio il display di una macchina utensile, o lo schermo di un telecomando). Questo schermo può mostrare **varie pagine** – come il telefono che cambia applicazione.

Quando in uno step vuoi **cambiare la pagina** mostrata da un display, usi:

```
view = main
```

Significa: "Mostra la pagina chiamata main sullo schermo". Le pagine disponibili sono definite nel file `objects.ini` (vedi capitolo dedicato).

Prendi il telecomando con la sua schermata principale

```
[step "Prendi il telecomando"]
description = Prendi il telecomando dal tavolo.
tool        = hand
hold        = pick remote at (-0.25, -0.07, 0.25) facing (0, 5, 0)
view        = main
```

[Immagine da inserire]

Diagramma a stati con tre cerchi nominati "main", "pumping", "idle" collegati da frecce con etichette degli step (es. "view = pumping"). Mostra le transizioni possibili tra le viste di un display.

Figura 16.2: Le transizioni tra viste: dichiarate in `objects.ini`, attivate dal tutorial con `view =`.

Capitolo 17

Animazioni 2D a frame (animation)

Per simulare **contatori**, **schermate che cambiano**, **sequenze ripetitive** puoi mostrare una serie di immagini sopra la scena 3D, una dopo l'altra. Si fa con:

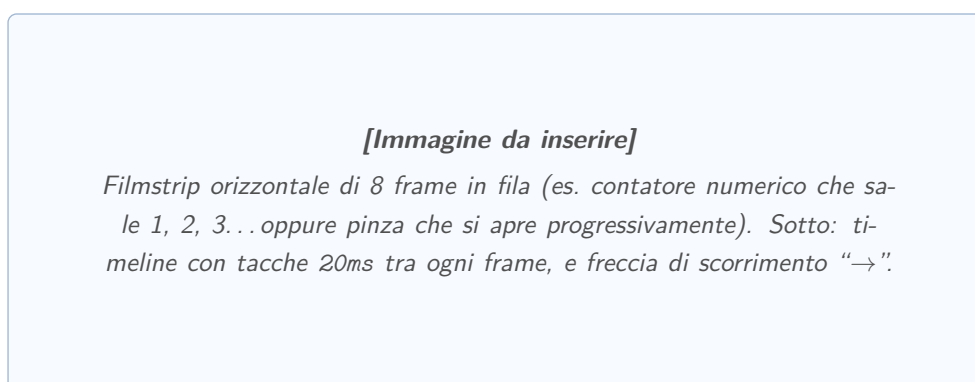


Figura 17.1: Un'animazione a frame: una sequenza di immagini mostrate ogni N millisecondi sopra la scena 3D.

```
animation = folder screens/mandrino at (60%, 60%) frame 20ms
```

Significa:

- **folder screens/mandrino** – la cartella dove ci sono le immagini (le carica tutte in ordine di nome)
- **at (60%, 60%)** – in che punto dello schermo (percentuali della finestra: 0%=sinistra/alto, 100%=destra/basso)
- **frame 20ms** – quanti millisecondi tra un'immagine e la prossima

Tutte e tre le parti sono opzionali (puoi metterne anche solo due).

[Immagine da inserire]

Screenshot della finestra dell'app con un riquadro semi-trasparente sovrapposto alla scena 3D, posizionato in alto-destra (al 60%, 60%). Linee guida tratteggiate mostrano le percentuali "60%" dall'angolo in alto a sinistra (X) e dall'alto (Y).

Figura 17.2: Posizionamento dell'animazione: percentuali sulla finestra (0% = sinistra/alto, 100% = destra/basso), non pixel.

Quasi sempre va con `element ...repeat N`

L'animazione è quasi sempre **guidata da clic ripetuti**: ogni clic fa avanzare di un frame. Nello stesso step si scrive l'`element` con `repeat N`:

```
[step "Apri e chiudi" highlight]
description = Apri e chiudi piu volte premendo il pulsante (4 cicli)
tool        = hand
element     = remote.Pulsante_r_unlock repeat 8
animation   = folder screens/mandrino at (60%, 60%) frame 20ms
```

Lo studente preme 8 volte il pulsante, ad ogni pressione l'immagine cambia, all'ottava lo step finisce.

Capitolo 18

I blocchi `[state]` e `[object]`

Oltre ai tre blocchi gerarchici (`[scene]`, `[section]`, `[step]`) visti nel Capitolo 4, esistono due blocchi **dichiarativi** che servono a descrivere oggetti con **più stati visivi** (per esempio una chiave che può essere “girata” o “non girata”) oppure modelli 3D **con parti interattive** (per esempio un pulpito di comando con i suoi pulsanti).

Per igiene di progetto questi blocchi **non si scrivono nel** `tutorial.cvtscript`, ma in un file separato chiamato `objects.ini` caricato automaticamente prima del tutorial.

18.1 `[state nome]` – Le varianti di un oggetto

Pensa a una **lampadina**: può essere **accesa** o **spenta**. Sono due **varianti** dello stesso oggetto. Si scrive così:

```
[state lampadina]
Variants = lampadina_off, lampadina_on
Default  = lampadina_off
```

Significa:

- Lo state si chiama `lampadina`
- Ha due varianti: `lampadina_off` e `lampadina_on` (sono i nomi dei mesh dentro al modello 3D)
- All’inizio è in `lampadina_off`

In uno step puoi cambiare variante con `after ::`

```
[step "Accendi" machine]
after : lampadina = lampadina_on
```

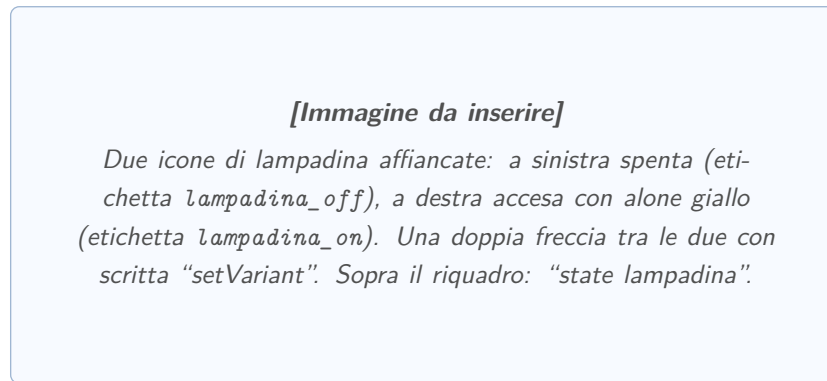


Figura 18.1: Uno state è un gruppo di varianti mutuamente esclusive: la lampadina è O accesa O spenta, mai entrambe.

18.2 [object nome] – Un oggetto con pulsanti dentro

Quando un modello 3D ha dei **pulsanti** che lo studente può cliccare (per esempio il pulpito di una macchina), lo dichiari così:

```
[object pulpito]
Model = models/pulpito.glb

InteractiveChild = Pulsante_mdi, button, onClick:setVariant:schermo=
    schermo002
InteractiveChild = chiave0, button, onClick:cycleVariant:chiave
```

Si legge:

- L'oggetto si chiama `pulpito` e usa il modello `models/pulpito.glb`
- Dentro c'è un pulsante `Pulsante_mdi`: quando lo clicchi, cambia la variante dello state `schermo` a `schermo002`.
- C'è anche `chiave0`: quando lo clicchi, cicla sulla **prossima variante** dello state `chiave` (`off` → `on` → `off`...).

A che servono?

Il file `objects.ini` **descrive una sola volta** che il pulpito ha 5 pulsanti, che lo schermo può mostrare 9 pagine, eccetera. Poi nel tutorial usi solo `element` = per attivarli e `after` : per cambiare le pagine.

È la stessa cosa che fai con le **persone**: prima dici "Mario è alto e ha i capelli neri" (= dichiarare l'oggetto), poi puoi parlare di Mario senza ridescriverlo ogni volta.

[Immagine da inserire]

Render del pulpito 3D con etichette colorate sui figli interattivi: Pulsante_mdi → setVariant:schermo=schermo002, chiave0 → cycleVariant:chiave, Pulsante_tool → setVariant:schermo=schermo003. Sopra: titolo "object pulpito".

Figura 18.2: Un object è un modello 3D con figli cliccabili: ogni InteractiveChild ha la sua azione onClick.

Capitolo 19

I file di configurazione attorno al tutorial

[Immagine da inserire]

Mappa orbitale: al centro un pianeta blu con etichetta "tutorial.cvtscript". In orbita 4 satelliti collegati da linee con frecce: homeconfig.ini (vetrina scenari), interfaceconfig.ini (mouse e camera), tool.ini (attrezzi), objects.ini (stati e pulsanti). Ogni satellite ha sotto una mini-etichetta che spiega il ruolo.

Figura 19.1: I cinque file che compongono uno scenario, e come si parlano tra loro.

Il file `tutorial.cvtscript` **non è solo**. Vive insieme a **quattro fratellini** che si trovano in altre cartelle. Ognuno ha un compito preciso:

File	A cosa serve
<code>scenes/homeconfig.ini</code>	Lista degli scenari disponibili (la pagina iniziale che vedi quando apri il programma)
<code>scenes/interfaceconfig.ini</code>	Personalizzazione dell'interfaccia (zoom, rotazione camera, sensibilità mouse)
<code>scenes/<NomeScenario>/Qualit�</code>	Qualit� che vede lo studente per questo scenario
<code>scenes/<NomeScenario>/Subject</code>	Subject oggetti interattivi (pulsanti, schermi) della scena

19.1 homeconfig.ini – La vetrina degli scenari

È la **lista dei tutorial** che vede lo studente quando apre il programma. Ogni scenario è un **blocco**, e dentro dichiara di che tutorial si tratta, quali modelli 3D caricare, dove mettere la telecamera all'inizio.

Esempio: una pagina con due scenari

```

title      = Campus Virtual Training Morbidelli X400
subtitle   = Seleziona uno scenario per cominciare

[Manutenzione pompa del vuoto]
CameraPos      = (-1.5, 0.5, -0.2)
CameraTarget    = (0, 0, 0)
AmbientLight    = 0x404040, 1.5
DirectionalLight = 0xffffffff, 2.5, (-8, 15, 10)
description     = Smontaggio e manutenzione della pompa Becker.
image          = menuimages/pompavuoto.png
tutorial        = scenes/Pompa_Becker/tutorial.cvtscript
tool           = scenes/Pompa_Becker/tool.ini
model          = models/culatta.glb
direction      = 0, 0, 1
model          = models/corpo.glb
direction      = 0, 0, 1

[Manutenzione Elettromandrino]
CameraPos      = (-1.11, 1.44, 3.97)
description     = Pulizia, lubrificazione e blocco/sblocco pinza.
image          = menuimages/1.png
tutorial        = scenes/Manutenzione_Elettromandrino/tutorial.
                 cvtscript
tool           = scenes/Manutenzione_Elettromandrino/tool.ini
model          = models/pulpito.glb
direction      = 0, 0, 1

```

Le righe importanti:

Riga	Cosa fa
<code>title / subtitle</code>	Titolo e sottotitolo della home page
<code>[Nome scenario]</code>	Apri il blocco dello scenario (= card visibile in home)
<code>description</code>	Testo descrittivo dello scenario
<code>image</code>	L'immagine di anteprima della card
<code>tutorial</code>	Il file <code>tutorial.cvtscript</code> da caricare
<code>tool</code>	Il file <code>tool.ini</code> da caricare per gli attrezzi
<code>CameraPos / CameraTarget</code>	Dove sta la telecamera all'apertura della scena
<code>AmbientLight</code>	Luce ambiente (colore esadecimale, intensità)
<code>DirectionalLight</code>	Luce principale (colore, intensità, direzione)
<code>model</code>	Un modello 3D da caricare nella scena (può apparire più volte)
<code>direction</code>	Per ogni <code>model</code> , la direzione di svita/estrazione

direction a che serve?

Quando scrivi `do : unscrew` su una vite, il programma deve sapere **in che direzione** esce. La risposta sta in **direction** di quel modello in `homeconfig.ini`: $(0, 0, 1)$ = esce verso Z+, $(-1, 0, 0)$ = esce verso X-, eccetera. Tu ti preoccupi solo di scrivere **direction** una volta per modello.

[Immagine da inserire]

Screenshot della home page reale dell'applicazione: titolo "Campus Virtual Training Morbidelli X400" in alto, sottotitolo, due card affiancate ("Manutenzione pompa del vuoto" con immagine pompa, "Manutenzione Elettromandrino" con immagine mandrino), pulsante "Avvia" su ognuna.

Figura 19.2: La home page generata da `homeconfig.ini`: ogni [Nome scenario] produce una card.

19.2 tool.ini – Gli attrezzi della cassetta

Per ogni scenario decidi **quali 4 attrezzi** vede lo studente nella barra in basso. È come scegliere gli attrezzi da mettere nello zainetto prima di partire.

Esempio: scenario Pompa Becker

```
[Tools]
Slot1 = brugola
Slot2 = chiave_inglese
Slot3 = mano
Slot4 = aria

[Tool:brugola]
Label          = Brugola
Icon           = utilimages/brugola.png
Cursor         = cursors/brugola_normal.svg
CursorPressed  = cursors/brugola_premuto_frame1.svg
CursorPressedFrame2= cursors/brugola_premuto_frame2.svg
CursorHotspotX = 4
CursorHotspotY = 9
Type           = tool
TutorialNames  = ChiaveBrugola, hex_key

[Tool:mano]
Label          = Mano
Icon           = utilimages/mano.png
```



```

Cursor      = cursors/mano_normal.svg
CursorPressed = cursors/mano_premuto_frame1.svg
CursorHotspotX= 16
CursorHotspotY= 16
Type        = tool
TutorialNames = Mani, hand

```

Spiegato facile facile:

Riga	Cosa fa
<code>[Tools]</code>	Il blocco con i 4 slot della barra attrezzi
<code>Slot1...Slot4</code>	Quale attrezzo va in ogni posizione (da sinistra a destra)
<code>[Tool:nome]</code>	Apre il blocco con i dettagli di quell'attrezzo
<code>Label</code>	Il nome che appare in italiano sotto l'icona
<code>Icon</code>	L'immagine che si vede nella barra
<code>Cursor</code>	Come diventa il cursore del mouse quando l'attrezzo è scelto
<code>CursorPressed / CursorPressedFrame2</code>	I due frame dell'animazione del cursore quando si clicca
<code>CursorHotspotX/Y</code>	Il puntino preciso del cursore (la "punta" dell'attrezzo)
<code>Type</code>	<code>tool</code> = è un attrezzo (al momento è l'unico valore)
<code>TutorialNames</code>	I nomi che il tutorial può usare nella riga <code>tool =</code> per attivarlo

Trucchetto

`TutorialNames` accetta **più nomi** separati da virgola. Così puoi scrivere indifferentemente `tool = hand` o `tool = Mani` e il programma sceglie comunque la mano.

[Immagine da inserire]

Barra strumenti reale dell'app con 4 slot in basso, ognuno etichettato sopra con "Slot1", "Slot2", "Slot3", "Slot4" (numerati da sinistra a destra). Sotto ogni slot l'icona dell'attrezzo: brugola, chiave inglese, mano, aria.

Figura 19.3: I 4 slot della barra strumenti: l'ordine in `tool.ini` è l'ordine sullo schermo, da sinistra a destra.

19.3 objects.ini – I pulsanti e gli schermi della scena

Qui dichiari i **blocchi dichiarativi** (`[state]` e `[object]`) che descrivono **cosa è cliccabile** nella scena 3D e **come si comporta** quando viene cliccato.

Esempio: pulpito di una macchina utensile

```
# La chiave puo essere girata o non girata
[state chiave]
Variants = chiave0, chiave1
Default = chiave0

# Lo schermo del pulpito puo mostrare diverse pagine
[state schermo]
Variants = schermo001, schermo002, schermo003, schermo004
Default = schermo001

# Il pulpito ha pulsanti e una chiave dentro
[object pulpito]
Model = models/pulpito.glb

InteractiveChild = chiave0, button, onClick:cycleVariant:chiave
InteractiveChild = chiave1, button, onClick:cycleVariant:chiave
InteractiveChild = Pulsante_mdi, button, onClick:setVariant:schermo=
    schermo002
InteractiveChild = Pulsante_tool, button, onClick:setVariant:schermo=
    schermo003
```

Cosa significa:

- Lo **state chiave** dice “c’è un mesh che può essere chiave0 oppure chiave1; all’inizio è chiave0”.
- Lo **state schermo** elenca le 4 pagine possibili dello schermo del pulpito.
- L’**oggetto pulpito** carica il modello 3D e dichiara che ha dei figli interattivi (i pulsanti).
- Per ogni pulsante **InteractiveChild** dice: “Quando lo clicchi, fai questa azione”.

Le azioni possibili al click:

Forma	Cosa fa
<code>onClick:setVariant:STATE=VARIANTE</code>	Imposta lo state alla variante data
<code>onClick:cycleVariant:STATE</code>	Va alla variante successiva (cicla)

Perché è separato dal tutorial?

Il pulpito è **sempre lo stesso** – ha gli stessi pulsanti – in tutti i tutorial. Se descrivessi i pulsanti dentro `tutorial.cvtscript`, dovrei ripetere la stessa cosa in ogni file. Mettendo le definizioni in `objects.ini` le scrivi **una volta sola**.

19.4 interfaceconfig.ini – Personalizzare i comandi

Questo file decide come si comporta il **mouse** (e il pinch-to-zoom su touch). Non serve modificarlo per scrivere un tutorial, ma è utile sapere che esiste.

Configurazione tipica

```
[ResetCameraButton]
Position = bottom-right

[CameraControls]
InvertVertical      = true
InvertHorizontal   = false
PinchSensitivity   = 0.001
InvertPinchZoom    = false
ScrollSensitivity  = 0.01
InvertScrollZoom   = false
ZoomMin            = 0.15
ZoomMax            = 1.3

[SpeechBubble]
DramaticAnimationDuration = 2000
```

Le impostazioni più importanti:

Riga	Cosa fa
<code>Position</code> (in <code>[ResetCameraButton]</code>)	Dove sta il pulsante di reset camera. Valori: top-right, top-left, top-center, bottom-right, bottom-left, bottom-center
<code>InvertVertical</code>	Inverte la rotazione su/giù della camera (true = naturale, false = invertito)
<code>InvertHorizontal</code>	Inverte la rotazione sinistra/destra
<code>PinchSensitivity</code>	Quanto è sensibile il pinch-to-zoom su touch (0.001–0.010, default 0.003)
<code>ScrollSensitivity</code>	Sensibilità della rotellina del mouse (0.010–0.060)
<code>ZoomMin / ZoomMax</code>	Quanto vicino e quanto lontano si può zoomare
<code>DramaticAnimationDuration</code>	Quanti millisecondi dura l'animazione di "ingrandimento drammatico" del fumetto quando ha la bandierina <code>highlight</code>

Quasi mai devi toccare `interfaceconfig.ini`. È pensato per piccoli ritocchi all'esperienza, non per il contenuto del tutorial.

Capitolo 20

Un esempio completo, riga per riga

Vediamo uno **step completo** e capiamolo riga per riga.

Step “Rimuovi la prima vite della culatta”

```
[step "Rimuovi la prima vite della culatta" highlight]
element      = vite_culatta_1
tool         = hex_key
description = Usa la chiave a brugola da 6mm per svitare la prima vite
              della culatta.
do           : unscrew distance 0.3
do           : place duration 0.2
```

Cosa succede quando lo studente arriva qui:

1. Apparirà un **fumetto** con scritto “Usa la chiave a brugola da 6mm...” (e il fumetto lampeggia, perché c’è la bandierina **highlight**).
2. Lo studente deve **scegliere la chiave a brugola** dalla barra degli strumenti.
3. Lo studente clicca sulla **vite** (vite_culatta_1).
4. La vite si **svita** girando e uscendo di 30 centimetri.
5. La vite si **appoggia a terra** in mezzo secondo.
6. Lo studente clicca la **freccia avanti** per andare al prossimo step.

[Immagine da inserire]

Sequenza di 4 fotogrammi orizzontali dello step "Rimuovi la prima vite":
 1) stato iniziale (vite avvitata, fumetto compare con *highlight*); 2) studente seleziona brugola, cursore cambia; 3) vite si svita (rotazione + traslazione); 4) vite appoggiata a terra accanto alla culatta. Numerazione 1–4 sotto.

Figura 20.1: Lo step "Rimuovi la prima vite" eseguito: cosa vede lo studente, momento per momento.

20.1 Step automatici in catena

Sequenza di viti consecutive

```
[step "Prima vite" highlight]
element      = vite_coperchio_1
tool         = hand
description  = Seleziona il tool "Mano" e rimuovi la prima vite del
               coperchio.
do           : unscrew
do           : place duration 0.2

[step "Seconda vite" auto]
element      = vite_coperchio_2
description  = Rimuovi la seconda vite del coperchio.
do           : unscrew
do           : place duration 0.2

[step "Terza vite" auto]
element      = vite_coperchio_3
description  = Rimuovi la terza vite del coperchio.
do           : unscrew
do           : place duration 0.2
```

Cosa succede:

- La **prima** vite la deve fare lo studente (perché ha *highlight*, niente *auto*).
- Dopo che ha tolto la prima vite con successo, la **seconda** e la **terza** si svitano **da sole** (*auto*).

Trucchetto

tool = *hand* lo scrivi **solo nella prima vite**. Le successive con *auto* lo **ereditano** dal contesto precedente.

20.2 Esempio di drag & drop

Step con trascinamento

```
[step "Monta il coperchio" highlight]
description = Posiziona il coperchio sopra il filtro.
tool        = hand
drag        = coperchio distance 0.3
```

Cosa succede:

1. Apparirà il fumetto “Posiziona il coperchio sopra il filtro”.
2. Lo studente seleziona la mano.
3. Lo studente **clicca e trascina** il coperchio.
4. Quando il coperchio è entro 30 cm dalla sua sede originale, **si aggancia automaticamente** lì.
5. Lo step viene completato e si va avanti.

20.3 Esempio di click su un pulsante 3D

Step con click su pulsante

```
[step "Vai alla schermata MDI" highlight]
description = Premi il pulsante MDI sul pulpito.
element     = pulpito.Pulsante_mdi
after       : schermo = schermo002
```

Cosa succede:

1. Apparirà il fumetto “Premi il pulsante MDI sul pulpito”.
2. Il pulsante MDI si **illumina** di giallo (auto-evidenziato).
3. Lo studente lo clicca.
4. Lo schermo del pulpito **cambia** mostrando la schermata “schermo002”.
5. Si passa al prossimo step automaticamente.

20.4 Esempio di click su una porta

Step con click su porta che si apre

```
[step "Apri la porta della cabina" highlight]
description = Apri la porta della cabina della macchina.
element     = a500.porta
do          : rotate around (0.615, 0, 0) by (0, 90, 0) duration 1.0
```

Cosa succede:

1. Apparirà il fumetto “Apri la porta della cabina della macchina”.
2. La porta si **illumina** di giallo per dire “cliccami”.
3. Lo studente la clicca.
4. La porta **ruota** di 90 gradi attorno al cardine in 1 secondo.
5. Quando l'animazione è finita, si passa al prossimo step.

Stessa keyword, comportamento diverso

Nei due esempi sopra abbiamo la stessa parola `element =`. Nel primo caso (pulsante) lo step ha `after :` e si passa subito al prossimo step dopo il click. Nel secondo (porta) lo step ha `do :` e si aspetta che l'animazione finisca. **Tu non devi dire al programma quale dei due casi è:** lui lo capisce dalle altre righe dello step.

Capitolo 21

Errori comuni e come evitarli

[Immagine da inserire]

Griglia 2×5 con dieci celle, ognuna con un simbolo × rosso e una parola chiave breve: “Maiuscolo”, “Virgolette”, “Numerare”, “glb”, “Italiano”, “Spazi”, “//”, “AutoSet”, “ActiveButtons”, “Animated...”. Sotto il titolo “i 10 errori più comuni”.

Figura 21.1: La mappa rapida dei dieci errori più comuni: usa questa pagina come check-list quando un tutorial non funziona.

21.1 Errore 1: scrivere le righe con la lettera maiuscola

```
Element=vite_coperchio_1  
Tool=Mani
```

```
element = vite_coperchio_1  
tool    = hand
```

Le maiuscole continuano a funzionare per **retrocompatibilità**, ma la sintassi nuova vuole tutto **minuscolo**.

21.2 Errore 2: dimenticare le virgolette nel nome dello step

```
[step Rimuovi la prima vite]
```

```
[step "Rimuovi la prima vite"]
```

Senza virgolette il programma si confonde perché ci sono spazi nel nome.

21.3 Errore 3: numerare le azioni manualmente

```
Action1 = unscrew  
Action2 = place(0.2)  
Action3 = move by (0, 0, -0.1)
```

```
do : unscrew  
do : place duration 0.2  
do : move by (0, 0, -0.1) duration 0.5
```

Tu scrivi `do :` e il programma le numera da solo. Se vuoi inserirne una in mezzo, basta aggiungere una riga – niente da rinumerare.

21.4 Errore 4: scrivere il nome del file .glb

```
element = models/vite.glb
```

```
element = vite
```

Il programma aggiunge da solo `models/` e `.glb`. Tu scrivi solo il **nome del pezzo**.

Eccezione: child interno a un modello

Se vuoi un pezzo dentro un altro (es. la “porta” dentro il modello “a500”), si scrive con il **punto**:

```
element = a500.porta
```

21.5 Errore 5: usare i nomi italiani degli attrezzi

```
tool = Mani  
tool = ChiaveBrugola
```

```
tool = hand  
tool = hex_key
```

I nomi italiani continuano a funzionare ma quelli inglesi sono più moderni.

21.6 Errore 6: dimenticare gli spazi attorno all’=

Il programma è abbastanza tollerante, ma per **leggibilità** scrivi sempre:

```
element = vite_coperchio_1  
tool    = hand
```

```
element=vite_coperchio_1  
tool=hand
```

21.7 Errore 7: mettere commenti con //

```
// Questo è un commento
```

```
| # Questo è un commento
```

Il # è il modo moderno. Il // ancora funziona ma il programma ti avvisa che è vecchio.

21.8 Errore 8: usare AutoSetVariant invece di after :

```
| [step "Scambio Utensile" machine]
| AutoSetVariant=tool=tool0
```

```
| [step "Scambio Utensile" machine]
| after : tool = tool0
```

after : si occupa da solo di cambiare la variante: se lo step ha un **element** = cliccabile cambia dopo il click, altrimenti cambia quando lo step parte.

21.9 Errore 9: usare ActiveButtons + AcceptTrigger a mano

```
| ActiveButtons=chiave0
| AcceptTrigger_Physical=pulpito.chiave0
| OnPhysicalTrigger=setVariant:chiave=chiave1;setVariant:schermo=
| schermo005
```

```
| element = pulpito.chiave0
| after : chiave = chiave1; schermo = schermo005
```

L'**element** dice "il pezzo da cliccare è la chiave", e **after :** dice "quando lo studente la clicca, applica questi cambi di stato". Le tre righe **ActiveButtons**, **AcceptTrigger_Physical** e **OnPhysicalTrigger** le aggiunge automaticamente il programma.

21.10 Errore 10: scrivere `button = ...` (vecchia versione del v3)

```
button = pulpito.Pulsante_mdi
after  : schermo = schermo002
```

```
element = pulpito.Pulsante_mdi
after   : schermo = schermo002
```

Nelle prime versioni del v3 c'era `button =` per i pulsanti 3D. Ora **tutto è `element`**: pulsanti, porte, leve, viti. Il programma capisce da solo cosa fare guardando le altre righe dello step. La parola `button =` continua a funzionare per non rompere i tutorial vecchi, ma il programma ti avvisa che è deprecata.

21.11 Errore 11: usare `Animated...` invece di `animation`

```
ActiveButtons=Pulsante_r_unlock
AcceptTrigger_Physical=remote.Pulsante_r_unlock
AnimatedImagesFolder=screens/mandrino
AnimatedPosition=(60%,60%)
AnimatedMaxTriggers=8
AnimatedFrameDelay=20
```

```
element  = remote.Pulsante_r_unlock repeat 8
animation = folder screens/mandrino at (60%, 60%) frame 20ms
```

`repeat 8` dice "8 click totali". `animation =` dice cosa mostrare nel frattempo.

Capitolo 22

Glossario delle parole importanti

Block (blocco)

Un pezzo di file racchiuso tra [e]. Esempi: `[scene]`, `[section "..."]`, `[step "..."]`, `[state ...]`, `[object ...]`.

Section (sezione, capitolo)

Un grande pezzo di tutorial (es. "Pulizia Filtro"). Contiene tanti step.

Step (passo)

Un singolo passo del tutorial. Lo studente vede il fumetto, fa l'azione, va al prossimo.

Element (elemento)

Il pezzo 3D coinvolto nello step. **È sempre la stessa parola**, qualunque sia il pezzo: una vite (`vite_coperchio_1`), un pulsante (`pulpito.Pulsante_mdi`), una porta (`a500.porta`), una leva. Il programma capisce dal contesto cosa fare quando lo studente lo clicca.

Tool (attrezzo)

L'attrezzo che lo studente deve scegliere. Cinque possibili: `hand`, `hex_key`, `wrench`, `air`, `spray`.

Description (descrizione)

Il testo che appare nel **fumetto** della spiegazione.

Action / do :

Un'azione che il pezzo fa nello step. Possono essercene tante in fila.

Camera

Gli "occhi" dello studente. Si dirige con `camera = position (...), target (...), ....`

Drag

Lo studente trascina il pezzo con il mouse fino alla destinazione.

After

Cosa succede subito dopo che lo studente clicca l'`element` (cambia uno o più stati con `after : nome_state = variante`).

Hold

Lo studente prende un oggetto in mano (es. un telecomando).

Bandierine

Parole speciali alla fine del nome di uno step. Tre esistono: `auto`, `machine`, `highlight`.

Coordinate (x, y, z)

Tre numeri che dicono “dove” nello spazio. X = destra/sinistra, Y = su/giù, Z = avanti/indietro.

Origin (suffisso _original)

Un riferimento alla **posizione iniziale** di un pezzo. Esempio: `tappino_grasso_dx_original` significa “la posizione dove sta il tappino di destra all'avvio”.

Companion (compagno)

Un pezzo che si muove **insieme** a quello principale dello step. Esempio: quando svitate la flangia, anche il tubo del grasso la segue.

Variant (variante)

Una “versione” diversa di un pezzo. Esempio: lo schermo del pulpito può essere `schermo001` (home) o `schermo002` (MDI). Cambiare variante significa mostrare una versione diversa.

State (stato)

Un gruppo di varianti possibili. Esempio: lo state schermo può essere uno di `[schermo001, schermo002, ...]`. Si dichiara nel blocco `[state nome]` dentro `objects.ini`.

Object (oggetto interattivo)

Un modello 3D con pulsanti e parti che lo studente può cliccare (es. il pulpito). Si dichiara nel blocco `[object nome]` dentro `objects.ini`.

Position / Rotation

Righe che spostano o ruotano un pezzo *prima* che lo step parta. Si scrivono in `[scene]`, in una `[section]` o dentro uno `[step]`.

Companion Un pezzo che si muove **insieme** a quello principale dello step. Può seguirlo passivamente (`follows master`) o fare il proprio movimento (`moves by`).

Repeat N (in element = X repeat N)

Lo studente deve cliccare l'`element` N volte. Si usa con `animation =` per animazioni a frame (apri/chiudi la pinza, sfoglia pagine, ecc.).

Animation Una sequenza di immagini 2D mostrate sopra la scena 3D. Avanza ad ogni clic del pulsante (vedi `repeat`).

View (vista di un display)

La pagina mostrata da uno schermo digitale 3D. Si cambia con `view = nome`. Le pagine possibili sono dichiarate in `objects.ini`.

Hold

Lo studente prende un oggetto in mano (`hold = pick`), continua a tenerlo (`hold = held`) o lo lascia (`hold = release`).

Message / Title / Video / Image

Mostra una finestra modale con testo, titolo, video o immagine. Lo studente chiude con OK e va avanti.

Conclusione

Adesso sai scrivere un tutorial. Ricorda i **tre segreti**:

1. **Scrivi minuscolo** (per la sintassi v3).
2. **Le virgolette** nei nomi di section e step.
3. **do** : per ogni azione, una per riga, in ordine.

Se ti dimentichi qualcosa, riapri questo manuale al **glossario** (Capitolo [22](#)) o all'**esempio completo** (Capitolo [20](#)).

Buon divertimento e buon tutorial!